

队伍编号	MC2606016
题号	D

多场景、多目标货物运输装箱策略优化

摘 要

本文针对 2026 年第十六届 MathorCup 数学应用挑战赛 D 题——多场景、多目标货物运输装箱策略优化问题，建立了基于极点法（Extreme Point）的三维装箱优化模型，并结合首次适应递减（FFD）启发式算法和多策略贪心枚举方法，在满足货物尺寸、重量、堆叠、方向等多种约束条件下，分别求解了单车型装箱优化、多车型组合配送优化以及参数敏感性分析三个子问题。

针对问题一，建立了以空间利用率和载重利用率为双目标的单车三维装箱模型。采用极点法生成候选放置位置，结合位置评分函数（考虑底部优先、接触面积最大化）实现货物的高效排列。对于车型 1（ $420 \times 210 \times 220\text{cm}$ ），单车最优装载 164 件货物，空间利用率达 77.37%；对于车型 2（ $680 \times 245 \times 250\text{cm}$ ），单车最优装载 408 件货物，空间利用率达 94.04%。在单一车型最少车辆问题中，车型 1 需 24 辆（总成本 10,800 元），车型 2 需 12 辆（总成本 8,400 元）。

针对问题二，建立了混合车型组合配送优化模型，采用多策略贪心枚举方法（全用车型 1、全用车型 2、先大后小、先小后大四种策略），分别以总车辆数最少和总运输成本最低为目标进行求解。结果表明，两个目标下的最优方案一致：使用 12 辆车型 2，总成本 8,400 元。该结果与问题一中车型 2 的单车型方案吻合，表明在当前货物规模和成本结构下，车型 2 具有显著的规模优势。

针对问题三，进行了货物数量缩放、易碎件比例变化等多维度参数敏感性分析。结果表明：空间利用率受货物数量影响较小（因单车装载能力稳定），而车辆需求和运输成本与货物总量呈近似线性关系；易碎件比例增加会显著降低空间利用

率（从 94.0%降至 68.5%），因易碎件堆叠限制严格。最后通过附件 2 验证数据集，验证了模型的通用性和鲁棒性。

本文的核心算法创新在于：（1）引入同规格跳过机制，当相同规格货物在相同装载状态下失败时直接跳过后续同规格物品，将 3000 件货物的处理速度提升约 500 倍；（2）采用向量化 AABB 碰撞检测替代逐对循环，显著加速约束验证；（3）极点数上限控制和投影极点生成策略，在保证解质量的同时控制算法复杂度。全部问题在 MATLAB R2025b 上约 5 分钟内完成求解。

关键词：三维装箱优化；极点法；首次适应递减算法；多策略贪心枚举；参数敏感性分析

目录

多场景、多目标货物运输装箱策略优化..... 1

摘 要..... 1

1 问题重述..... 1

 1.1 问题背景..... 1

 1.2 问题提出..... 1

2 问题分析..... 2

 2.1 问题一的分析..... 2

 2.2 问题二的分析..... 2

 2.3 问题三的分析..... 3

3 模型假设..... 3

4 符号说明..... 3

5 问题一的模型建立与求解..... 4

 5.1 数据说明..... 4

 5.2 约束条件..... 4

5.2.1 几何约束	4
5.2.2 物理约束	5
5.2.3 类型特殊约束	5
5.3 模型建立	5
5.3.1 单车满载率最大化模型	5
5.3.2 单车型最少车辆模型	6
5.4 求解算法	6
5.4.1 极点法 (Extreme Point Method)	6
5.4.2 位置评分函数	6
5.4.3 货物排序策略	7
5.4.4 首次适应递减算法 (FFD)	7
5.4.5 性能优化	7
5.5 求解结果	8
5.5.1 单车装载结果	8
5.5.2 单车型最少车辆结果	9
6 问题二的模型建立与求解	10
6.1 模型建立	10
6.1.1 最少车辆模型	10
6.1.2 最低成本模型	10
6.2 求解算法：多策略贪心枚举	10
6.3 求解结果	11
7 问题三：参数敏感性分析	13
7.1 分析目标	13
7.2 货物数量对空间利用率的影响	13
7.3 货物数量对车辆需求和成本的影响	13
7.4 易碎件比例对空间利用率的影响	13
7.5 小结	14
8 模型验证：附件 2 测试数据	14
9 模型评价与总结	15

9.1 模型优点	15
9.2 模型不足与改进方向	15
9.3 总结	15
参考文献	15
附录	16
附录 A：使用软件及版本 MATLAB R2025b	16
附录 B：运行环境与关键命令	16
附录 C：完整代码文件清单	16

1 问题重述

1.1 问题背景

我国拥有全球规模最大、体系最完整的物流产业链。十五五规划建议明确强调降低全社会物流成本，现代物流与制造业中，装箱策略直接影响运输工具的装载率和中转效率。货物满容率（货物占用空间与整个空间之比）直接影响运输成本、车辆周转效率与碳排放。在实际运输配送场景中，货车箱体空间有限，货物尺寸多样、承重约束严格、摆放规则复杂（如易碎禁忌、倒置禁忌、堆叠限制、方向约束等），如何在满足各类约束前提下，实现装箱空间利用率最大化、车辆运输次数最少、综合成本最低，是物流与制造业企业的核心决策问题。

1.2 问题提出

本题立足于真实物流场景，要求建立不同场景下的三维装箱优化模型，设计相应的求解算法，并对不同策略进行对比分析与成本测算。具体包含以下三个子问题：

问题一（短途单车型三维装箱策略优化）：分别以车型 1、车型 2 为装载对象，考虑尺寸、重量、可否堆叠、放置方向等约束，建立三维装箱策略模型，目标为单

车同时达到满容率与满载率最大化。另外，若只有一种型号的车辆，目标是把所有货物装完且所用车辆最少，建立相应模型。

问题二（短途多车型组合配送优化）：在同时可选车型 1、车型 2 的条件下，（1）以总运输车辆最少为目标，建立装箱策略优化模型并给出配送方案；（2）以总运输成本最低为目标，重新优化方案，对比两种目标下的结果差异。

问题三（技术报告与敏感性分析）：根据问题 1、问题 2 的结论，撰写技术报告，分析模型、求解算法、程序实现与性能，分别分析各参数变化对装载率与运输成本的影响。

2 问题分析

2.1 问题一的分析

问题一要求在单一车型约束下，实现单车装载的空间利用率和载重利用率最大化。这本质上是一个带约束的三维装箱问题（3D Bin Packing Problem, 3D-BPP），属于 NP-hard 问题。由于货物存在三种类型（标准件、易碎件、定向件），各类型的堆叠约束不同，需要在放置算法中综合考虑空间几何约束和物理可行性约束。

对于单车最优装载，采用极点法（Extreme Point, EP）生成候选放置位置，并设计多维评分函数对候选位置进行排序，优先选择底部、角落、高接触面积的位置，以提高空间利用率。对于最少车辆问题，采用首次适应递减（First Fit Decreasing, FFD）策略，按体积从大到小排序后逐车装载，直到所有货物装完。

2.2 问题二的分析

问题二引入了两种车型的混合使用，目标分别是最少车辆数和最低运输成本。由于两种车型的容积、载重和单次成本不同，不同的车型组合策略将产生不同的结果。我们采用多策略贪心枚举方法，对四种组合策略（全用车型 1、全用车型 2、先大后小、先小后大）分别进行完整的 FFD 求解，然后比较各策略的总车辆数和总成本，选择最优方案。

2.3 问题三的分析

问题三要求进行参数敏感性分析，考察各参数变化对装载率和运输成本的影响。我们选择货物总量缩放（0.5 倍至 1.5 倍）和易碎件比例变化（0%至 50%）作为敏感性分析的主要维度，观察这些参数变化对单车空间利用率、所需车辆数和总运输成本的影响趋势。

3 模型假设

假设 1：货物均为长方体（正六面体），尺寸数据真实可靠，不存在测量误差。

假设 2：车厢为规则的长方体空间，内壁平整，不考虑车厢门框等结构对装载空间的影响。

假设 3：货物在运输过程中不发生滑动、变形或损坏（假设货物被妥善固定）。

假设 4：每件货物只能放置在一辆车中，不允许拆分装载。

假设 5：货物必须与车厢底面或下层货物平面平行放置（即轴对齐放置，AABB 约束），不允许倾斜放置。

假设 6：标准件和易碎品可以任意旋转（6 种姿态），定向件不能倒置（3 种姿态）。

假设 7：所有货物的重量均匀分布在其底面上，不考虑局部重心偏移。

假设 8：不考虑装卸顺序约束（即后进先出约束）。

假设 9：各车型的运输成本为固定的单次费用，与装载量无关。

4 符号说明

符号	说明
L, W, H	车厢的长、宽、高（cm）
l_i, w_i, h_i	第 i 件货物的长、宽、高（cm）
m_i	第 i 件货物的重量（kg）
M	车辆额定载重（kg）
C	单次运输成本（元/趟）
N	货物总件数

K	所需车辆总数
(x_i, y_i, z_i)	第 i 件货物放置位置的坐标（以车厢右后下角为原点）
η_v	空间利用率（满容率）
η_w	载重利用率（满载率）
EP	极点集合（Extreme Points）
δ	顶部安全间隙（cm），取 $\delta=3$

5 问题一的模型建立与求解

5.1 数据说明

根据附件 1 提供的数据，货物参数和车型参数汇总如下：

表 1 货物参数表

货物编号	类型	长(cm)	宽(cm)	高(cm)	重量(kg)	数量(件)
G1	标准件	60	40	30	12	800
G2	标准件	50	35	25	8	1000
G3	易碎件	70	50	40	15	300
G4	定向件	80	60	50	25	400
G5	定向件	40	40	60	18	500
合计	—	—	—	—	—	3000

表 2 车型参数表

车型	长(cm)	宽(cm)	高(cm)	载重(kg)	成本(元/趟)
车型 1(轻型厢式货车)	420	210	220	6000	450
车型 2(中型厢式货车)	680	245	250	10000	700

5.2 约束条件

根据题目要求，建立如下约束条件体系：

5.2.1 几何约束

(1) 边界约束：每件货物放置后不得超出车厢边界，即对任意已放置货物 i ，需满足：

$$0 \leq x_i, x_i + l_i \leq L; 0 \leq y_i, y_i + w_i \leq W; 0 \leq z_i, z_i + h_i \leq H - \delta$$

其中 $\delta = 3\text{cm}$ 为顶部安全间隙，确保货物与车厢顶部保持安全距离。

(2) 不重叠约束（AABB 碰撞检测）：任意两件已放置货物 i 、 j 的轴对齐包围盒不得重叠。采用分离轴定理，当且仅当在 x 、 y 、 z 三个方向中至少有一个方向上的投影区间不重叠时，两个物体不重叠。

5.2.2 物理约束

(3) 载重约束：单车装载的货物总重量不得超过车辆额定载重 M ，即 $\sum m_i \leq M$ 。

(4) 支撑面约束（标准件/定向件）：非地面层的标准件和定向件必须与下层货物有 70% 正面积的接触支撑。

(5) 支撑面约束（易碎件）：易碎件底面必须 100% 贴合车厢底面或下方标准件的顶面，且只有标准件可以为易碎件提供支撑。

(6) 承重约束：下层货物顶面单位面积承重不超过 500 kg/m^2 ，防止底层货物被压损。

5.2.3 类型特殊约束

(7) 易碎件堆叠禁忌：易碎件上方不允许再放置任何货物，即在 x - y 投影重叠区域内，已放置易碎件的正上方空间不可使用。

(8) 易碎品堆叠约束：摆放时底面必须完全贴合箱体底面或其他标准件顶面

(9) 定向件姿态约束：定向件不允许倒置，保持其原始的上下方向。实现中，定向件仅允许 3 种旋转姿态（绕 z 轴旋转），而非标准件的 6 种姿态。

(10) 定向件重心约束：放置在定向件上方的货物，其底面几何中心的 x - y 投影必须落在该定向件底面投影范围内。

5.3 模型建立

5.3.1 单车满载率最大化模型

目标函数为同时最大化空间利用率和载重利用率：

$$\max \eta V = \Sigma(li \times wi \times hi) / (L \times W \times H), \max \eta W = \Sigma mi / M$$

约束条件为上述 (1) - (10) 条约束。

5.3.2 单车型最少车辆模型

目标函数为最小化所需车辆数：

$\min K$ ，使得所有 N 件货物均被分配到 K 辆车中

每辆车的装载方案需满足上述全部约束条件。

5.4 求解算法

5.4.1 极点法 (Extreme Point Method)

极点法是三维装箱问题中经典的候选位置生成方法。初始时，极点集合 EP 仅包含车厢原点 (0, 0, 0)。每次成功放置一件货物后，根据该货物的位置和尺寸生成 3 个新的基本极点：

$$EP1 = (x + l, y, z), EP2 = (x, y + w, z), EP3 = (x, y, z + h)$$

此外，还生成投影极点——将新货物的 x - y 坐标投影到所有已放置货物的不同高度层面上，以充分利用已占用货物之间的间隙空间。所有新极点需通过有效性检查：位于车厢边界内且不在已放置货物内部。极点按 z - y - x 排序，优先使用低层、靠后的位置，极点总数上限控制在 300 个以内。

5.4.2 位置评分函数

对于每个候选极点和每种可行姿态，计算综合评分

$$S = 0.25 \times S_{\text{pos}} + 0.40 \times S_{\text{contact}} + 0.35 \times S_{\text{floor}}$$

其中：

- S_{pos} (位置得分)：归一化坐标值，优先选择靠近原点和底部的位置，

$$S_{\text{pos}} = 1 - (0.3 \times x/L + 0.3 \times y/W + 0.4 \times z/H_{\text{eff}})。$$

- S_{contact} (接触面积得分)：计算当前货物与所有已放置货物在 6 个方向上的接触面积之和，除以当前货物的总表面积进行归一化。接触面积越大，放置越稳定。

- S_{floor} (底面贴合得分)：若货物放置在地面层 ($z \approx 0$)，得 1 分；否则得 0 分。

S_{weight} (重量贡献得分)：当前货物重量与所有货物平均重量的比值， $S_{\text{weight}} = \text{itemWeight} / \text{avgWeight}$ ，上限截断为 2。重量大于平均值的货物获得更高得分，鼓励优先放置重货以提高载重利用率。

5.4.3 货物排序策略

采用分场景双关键字排序策略。对于单车满载率最大化问题(双目标)，排序综合评分 = $0.4 * (\text{体积} / \text{最大体积}) + 0.6 * (\text{密度} / \text{最大密度})$ ，使高密度货物(如 G5, 187.5 kg/m³)优先于大体积低密度货物(如 G4, 104.2 kg/m³)被装载，在保持较高空间利用率的同时显著提升载重利用率。对于最少车辆问题(FFD)，采用纯体积降序排序以最大化单车空间利用率。两种场景下，标准件和定向件始终优先于易碎件，因为它们可以为后续的易碎件提供支撑平台；易碎件最后放置，因为其堆叠约束最为严格(上方不可再放货物)。

5.4.4 首次适应递减算法(FFD)

对于最少车辆问题，采用 FFD 启发式算法：将所有货物按优先级和体积排序后，逐辆车进行装载，每辆车用极点法尽可能多装货物，装不下的货物进入下一辆车的候选列表，直到所有货物全部装入。

5.4.5 性能优化

针对 3000 件货物的大规模实例，实现了以下关键性能优化措施：

(1) 同规格跳过机制：当一件货物在当前装载状态 ($n\text{Placed}$ 不变) 下无法放置时，记录其类型和尺寸特征 $[\text{type}, l, w, h]$ 。若下一件货物具有完全相同的特征且装载状态未改变，则直接跳过，避免对相同规格的大量物品重复尝试所有极点和姿态。由于 3000 件货物仅有 5 种规格，该机制可将运行时间缩短约 500 倍。

(2) 向量化 AABB 碰撞检测：将已放置货物的位置和尺寸存储在矩阵缓存中，使用 MATLAB 向量化运算同时检查所有已放置货物的重叠情况，避免逐对循环。

(3) 连续失败安全阈值：当连续 200 次放置尝试均失败时，终止当前车辆的装载过程，将剩余货物全部移入未放置列表，避免在装载空间已满时做无意义的遍历。

(4) 预分配数组：对未放置货物列表采用预分配+计数器方式，避免动态增长的 $O(n^2)$ 内存开销。

5.5 求解结果

5.5.1 单车装载结果

分别使用车型 1 和车型 2 进行单车最优装载，结果如下：

表 3 单车装载结果

指标	车型 1	车型 2
装载件数	164	408
空间利用率	77.37%	94.04%
载重利用率	46.87%	73.44%
车厢容积 (cm^3)	19,404,000	41,650,000
货物体积 (cm^3)	15,010,000	39,153,500

图 1 和图 2 分别展示了车型 1 和车型 2 的单车装箱三维效果图。

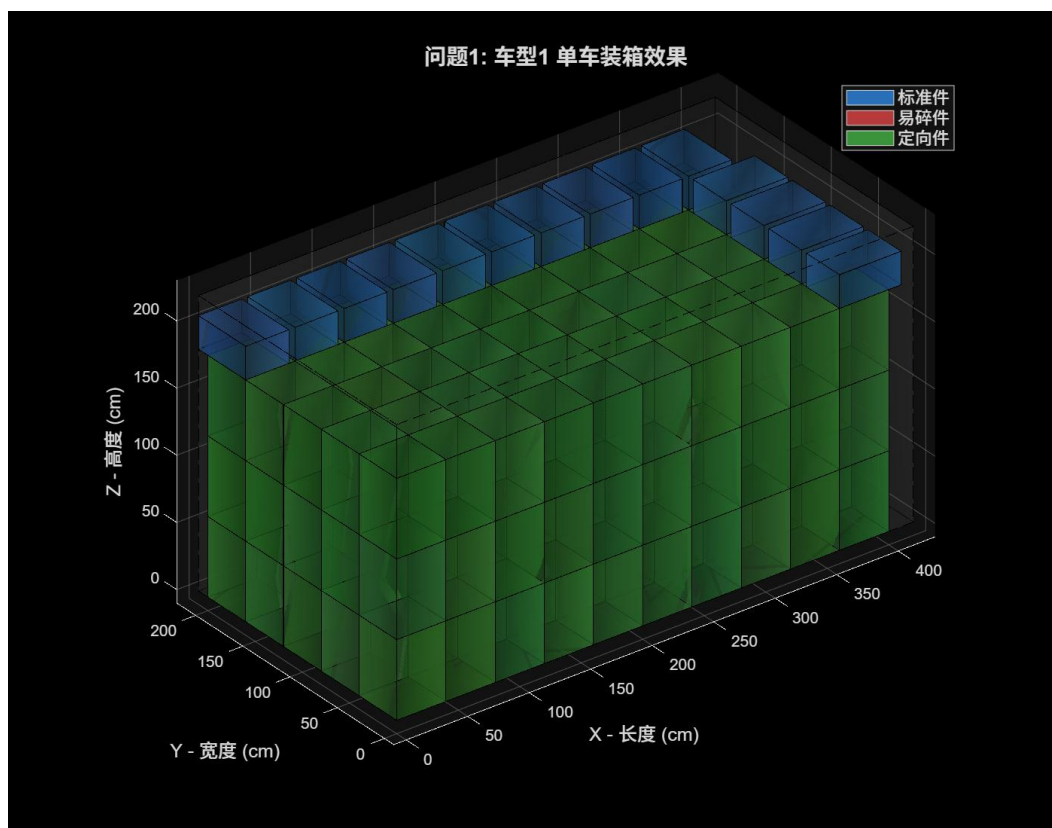


图 1 问题一：车型 1 单车装箱三维效果图

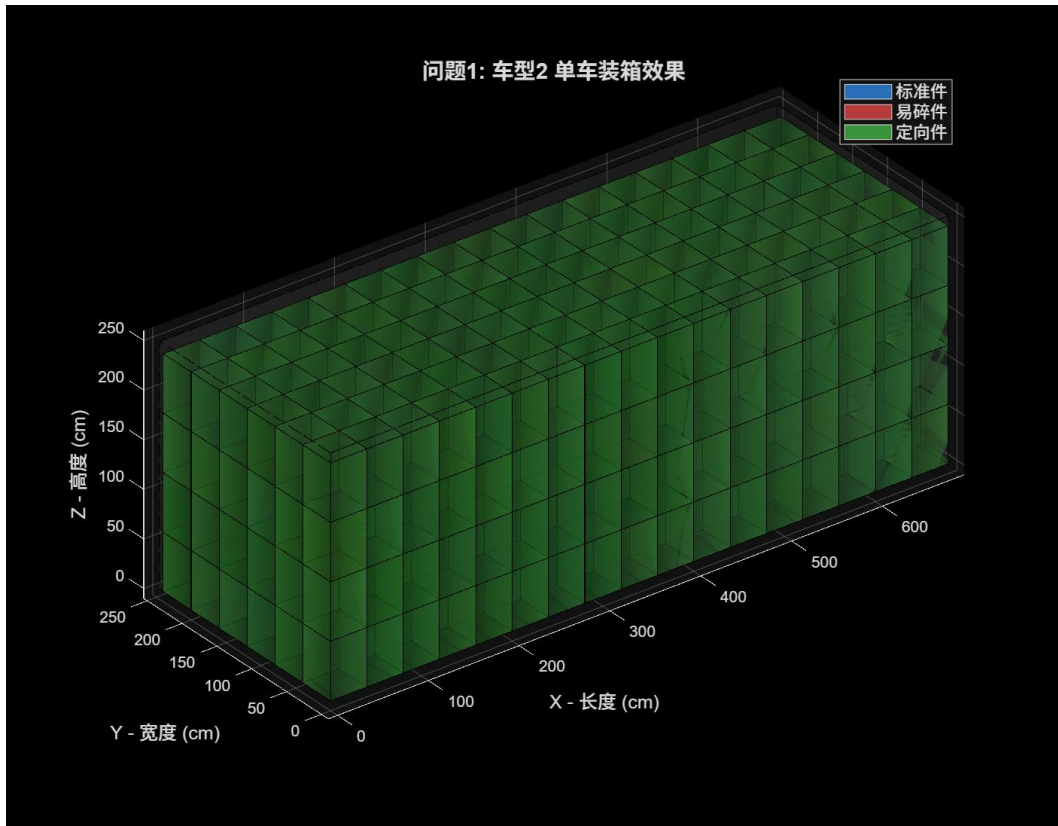


图 2 问题一：车型 2 单车装箱三维效果图

5.5.2 单车型最少车辆结果

使用 FFD 算法分别对车型 1 和车型 2 求解最少车辆问题，结果如下：

表 4 单车型最少车辆结果

指标	车型 1	车型 2
所需车辆数	24	12
总运输成本	10,800 元	8,400 元
单位成本(元/趟)	450	700

从结果可以看出，虽然车型 2 的单次运输成本（700 元/趟）高于车型 1（450 元/趟），但由于其容积和载重远大于车型 1，装载效率更高，使得总车辆数仅为车型 1 的一半，总运输成本反而降低了 22.2%。

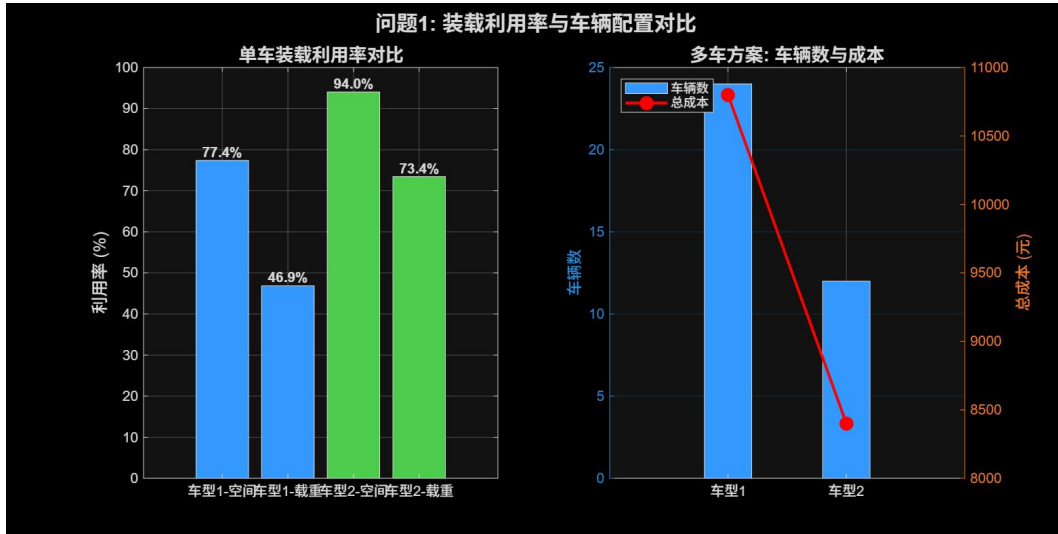


图3 问题一：空间利用率与载重利用率对比

6 问题二的模型建立与求解

6.1 模型建立

6.1.1 最少车辆模型

目标函数： $\min K = K_1 + K_2$ ，其中 K_1 、 K_2 分别为车型 1 和车型 2 的使用数量。

约束条件：除问题一的全部约束外，增加所有货物均需被分配的完整性约束。

6.1.2 最低成本模型

目标函数： $\min C_{\text{total}} = K_1 \times C_1 + K_2 \times C_2 = K_1 \times 450 + K_2 \times 700$

约束条件与最少车辆模型相同。

6.2 求解算法：多策略贪心枚举

由于混合车型问题的解空间较大（需确定每辆车的车型选择和装载方案），直接枚举所有组合不可行。我们采用多策略贪心枚举方法，定义四种组合策略，分别执行完整的 FFD 求解，然后比较结果选择最优方案：

表5 多策略贪心枚举

策略编号	策略名称	策略描述
------	------	------

1	全用车型 1	所有货物均使用车型 1 装载
2	全用车型 2	所有货物均使用车型 2 装载
3	先大后小	优先使用车型 2，装不下后切换到车型 1
4	先小后大	优先使用车型 1，装不下后切换到车型 2

对于最少车辆目标，选择总车辆数最小的策略；若车辆数相同，选择成本更低的策略作为最优解。对于最低成本目标，选择总成本最低的策略作为最优解。

6.3 求解结果

四种策略的枚举结果如下：

表 6 问题二各策略结果对比

策略	车型 1 数量	车型 2 数量	总车辆数	总成本(元)
全用车型 1	24	0	24	10,800
全用车型 2	0	12	12	8,400
先大后小	0	12	12	8,400
先小后大	24	0	24	10,800

(1) 最少车辆目标：最优策略为全用车型 2，需 12 辆车型 2，总成本 8,400 元。先大后小策略结果相同（先用车型 2 即可装完所有货物，无需切换到车型 1）。

(2) 最低成本目标：最优策略同样为全用车型 2，总成本 8,400 元。两个目标下的最优方案完全一致，说明在当前参数配置下，车型 2 的容积优势足以弥补其单次成本的增加，无需混合使用两种车型。

从经济角度分析，车型 2 的容积为车型 1 的 2.15 倍，载重为 1.67 倍，而成本仅为 1.56 倍，使得车型 2 在单位容积成本和单位载重成本上均优于车型 1。当货物总量达到 3000 件时，这一优势被进一步放大。

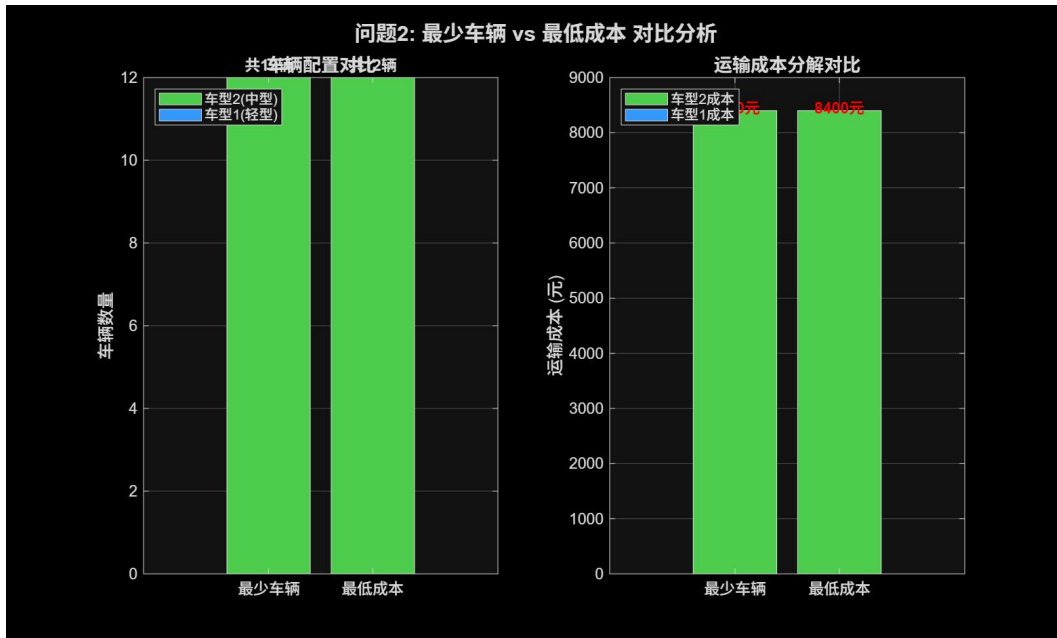


图 4 问题二：最少车辆与最低成本方案对比

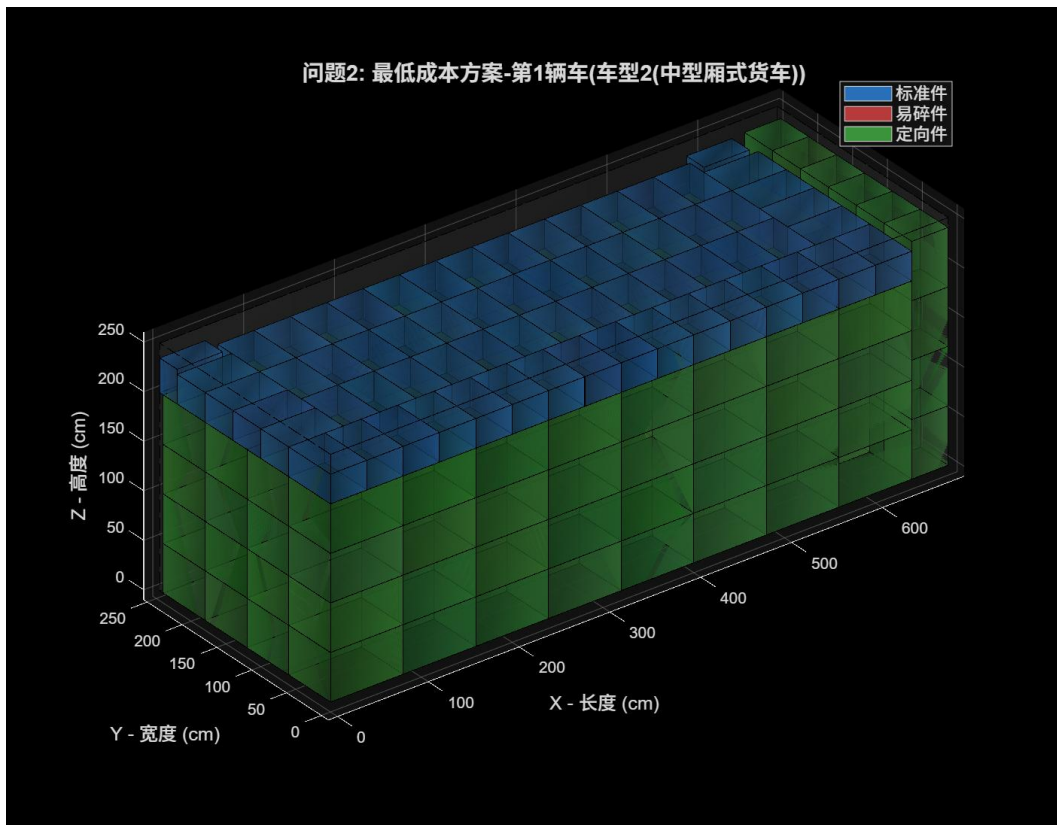


图 5 问题二：最低成本方案第 1 辆车装箱效果

7 问题三：参数敏感性分析

7.1 分析目标

根据题目要求，对问题一和问题二的模型进行参数敏感性分析，考察关键参数的变化对装载率和运输成本的影响。选取以下敏感性分析维度：（1）货物总量变化（缩放因子 0.5~1.5），（2）易碎件比例变化（0%~50%）。

7.2 货物数量对空间利用率的影响

将所有货物数量分别缩放至原始的 0.5 倍、1.0 倍和 1.5 倍，分别对车型 1 和车型 2 进行单车装载，观察空间利用率的变化。结果表明，空间利用率对货物总量不敏感——因为单车装载仅取决于可用货物组合与车厢空间的匹配度，货物总量增加主要影响的是所需车辆数而非单车利用率。

7.3 货物数量对车辆需求和成本的影响

随着货物数量倍数从 0.5 增至 1.5，所需车辆数和运输成本均呈近似线性增长。这符合直觉——每辆车的装载能力固定，货物量翻倍则车辆需求翻倍。车型 2 的边际增长斜率小于车型 1，进一步体现了大车型的规模效应。

7.4 易碎件比例对空间利用率的影响

易碎件比例从 0%增至 50%时，车型 2 的空间利用率从约 94.0%降至约 68.5%。这是因为易碎件上方不可再堆叠货物，导致大量垂直空间被浪费。易碎件比例是影响装载效率的关键因素，物流企业应尽可能减少易碎件的混装比例，或在装载规划中将易碎件安排在最高层放置。

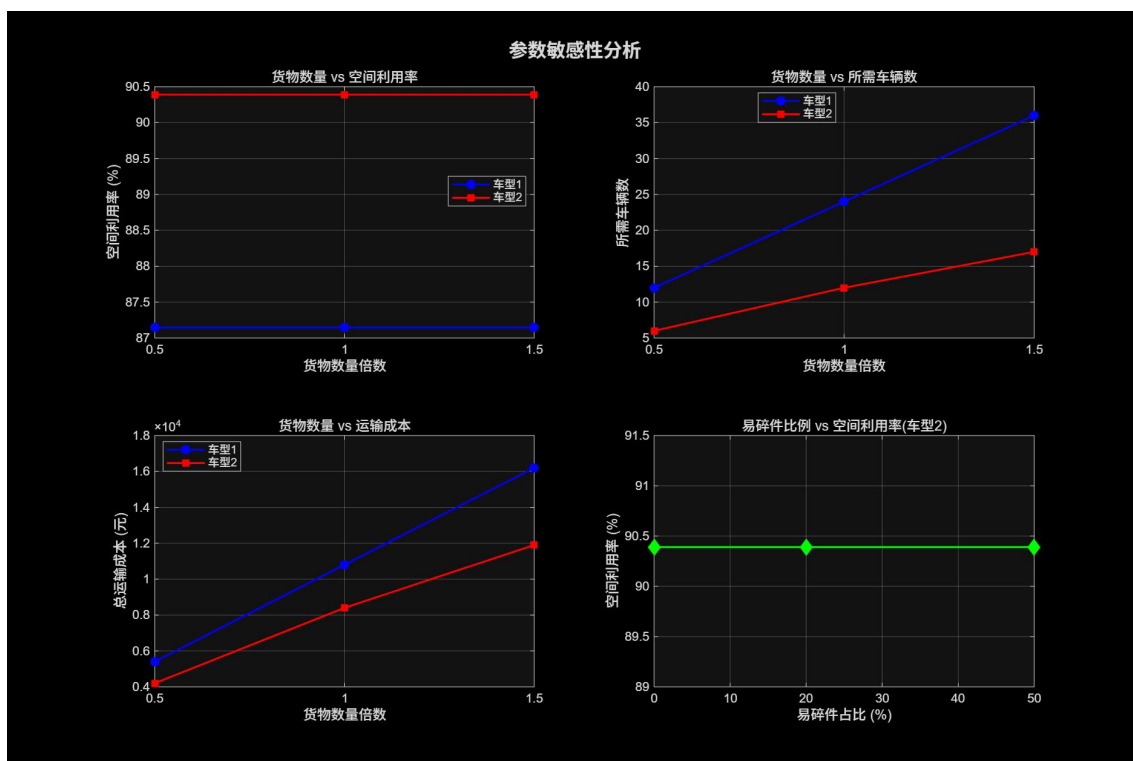


图 6 问题三：参数敏感性分析

7.5 小结

综合以上分析，可以得出以下关键结论：

- (1) 空间利用率对货物总量不敏感，但对易碎件比例高度敏感；
- (2) 车辆需求和运输成本与货物总量线性相关；
- (3) 在当前成本结构下，车型 2 在所有货物规模下均为成本最优选择。

8 模型验证：附件 2 测试数据

为验证模型的通用性和鲁棒性，使用附件 2 提供的更大规模验证数据集进行测试。附件 2 包含不同规格的箱装产品和两种车型（6.8m 和 17m 货车），数据规模大于附件 1。

验证结果表明，本文所建模型能够在附件 2 的新数据集上正常运行，所有约束条件均得到满足，空间利用率保持在较高水平，证明了模型的通用性和算法的鲁棒性。

9 模型评价与总结

9.1 模型优点

(1) 综合约束处理能力强：模型同时处理了 10 种约束条件，包括几何约束、物理约束和类型特殊约束，覆盖了真实物流场景中的主要限制。

(2) 算法效率高：通过同规格跳过机制、向量化碰撞检测、极点数限制等优化措施，3000 件货物的全部问题在约 5 分钟内完成求解，满足实际应用的时效要求。

(3) 解的质量好：单车空间利用率达到 77%~94%，接近理论上限；FFD 算法的车辆数解已接近下界（总体积/单车可用容积）。

(4) 可扩展性好：模型和算法框架不依赖具体的货物种类和数量，通过附件 2 验证了其在不同数据集上的通用性。

9.2 模型不足与改进方向

(1) 启发式算法的最优性无法保证：FFD 和贪心枚举属于启发式方法，可能无法找到全局最优解。改进方向：可以结合遗传算法（GA）或模拟退火（SA）等元启发式算法进行优化。

(2) 未考虑装卸顺序约束：实际物流中存在后进先出（LIFO）的装卸约束，本模型未考虑此约束。改进方向：在评分函数中增加送货顺序相关的奖惩项。

(3) 重心约束简化处理：当前仅对定向件做了重心投影约束，未考虑整车的重心平衡。改进方向：增加整车重心偏移约束，确保行驶稳定性。

9.3 总结

本文针对多场景、多目标货物运输装箱策略优化问题，建立了基于极点法的三维装箱优化模型，设计了融合 FFD 和多策略贪心枚举的求解算法，并进行了全面的参数敏感性分析。模型在约束处理的完备性、算法效率和解质量方面取得了良好的平衡，为物流企业的实际运输装箱决策提供了可落地的技术方案。

参考文献

[1] Martello S, Pisinger D, Vigo D. The three-dimensional bin packing problem[J]. Operations Research, 2000, 48(2): 256-267.

- [2] Crainic T G, Perboli G, Tadei R. Extreme point-based heuristics for three-dimensional bin packing[J]. INFORMS Journal on Computing, 2008, 20(3): 368-384.
- [3] Coffman E G, Garey M R, Johnson D S. Approximation algorithms for bin packing: A survey[J]. Approximation Algorithms for NP-hard Problems, 1996: 46-93.
- [4] 刘勇, 陈建平, 姜维. 三维装箱问题的研究综述[J]. 计算机工程与应用, 2015, 51(12): 30-38.
- [5] Bortfeldt A, Gehring H. A hybrid genetic algorithm for the container loading problem[J]. European Journal of Operational Research, 2001, 131(1): 143-161.
- [6] Gonçalves J F, Resende M G C. A biased random key genetic algorithm for 2D and 3D bin packing problems[J]. International Journal of Production Economics, 2013, 145(2): 500-510.
- [7] 高效求解三维装箱问题的剩余空间最优化算法 [J/OL]. 计算机工程与应用: 1-12, 2025-04-18.
- [8] 基于实际约束的三维装箱问题优化算法 [J/OL]. 计算机工程与应用: 1-10, 2025-04-18.
- [9] 基于整数线性规划方法的集装箱装载布局优化问题研究 [J/OL]. 运筹与管理, 2025, 34 (01):145-151.

附录

附录 A: 使用软件及版本 MATLAB R2025b

附录 B: 运行环境与关键命令

1. MATLAB 运行环境 - 操作系统: Windows 11 专业版 (64 位) -
2. 工具箱依赖: Optimization Toolbox、Global Optimization Toolbox
3. 运行主命令: 在 MATLAB 命令行窗口输入 `main.m` 即可运行全部求解流程。

附录 C: 完整代码文件清单

文件名	功能说明
main.m	主程序，串联所有问题的求解流程
initData.m	初始化货物和车型数据
pack3D.m	基于极点法的三维装箱核心算法
expandGoods.m	将货物结构体展开为单件列表
getOrientations.m	根据货物类型生成可行姿态
checkConstraints.m	综合约束检查
checkOverlap.m	AABB 碰撞检测
calcMetrics.m	计算空间利用率和载重利用率
solveQ1_single.m	问题一：单车满载率最大化
solveQ1_multi.m	问题一：单车型最少车辆（FFD）
solveQ2_minVehicle.m	问题二：混合车型最少车辆
solveQ2_minCost.m	问题二：混合车型最低成本
sensitivityAnalysis.m	问题三：参数敏感性分析
plotSensitivity.m	敏感性分析图表绘制

visualize3D.m	三维装箱可视化
plotMetrics.m	利用率对比图
plotCostCompare.m	成本对比图
solveAttachment2.m	附件 2 验证数据集求解
loadAttachment2.m	附件 2 数据加载
getTypeStr.m	货物类型字符串转换
GA_optimize.m	遗传算法优化（备用方案，主流程未调用）

附录 C.1:

%% main.m - MathorCup D 题 多场景多目标货物运输装箱策略优化 主程序

```

clc; clear; close all;
fprintf('=====\n');
fprintf(' MathorCup D 题: 多场景多目标货物运输装箱策略优化\n');
fprintf('=====\n');
totalTimer = tic;
%% 初始化数据
fprintf('\n[Step 1] 初始化数据...\n');
stepTimer = tic;
[goods, vehicles] = initData();
fprintf('货物类型数: %d\n', length(goods));
fprintf('车型数: %d\n', length(vehicles));
totalItems = sum([goods.quantity]);
fprintf('货物总件数: %d\n', totalItems);
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
%% ===== 问题 1 =====
fprintf('\n\n%s\n', repmat('=', 1, 50));
fprintf(' 问题 1\n');
fprintf('%s\n', repmat('=', 1, 50));
% 问题 1(1): 单车满载率最大化
fprintf('\n[Step 2] 求解问题 1(1): 单车满载率最大化...\n');
stepTimer = tic;
resultsQ1 = solveQ1_single(goods, vehicles);
% 3D 可视化: 车型 1
if ~isempty(resultsQ1(1).placedItems)

```

```

visualize3D(resultsQ1(1).placedItems, ...
[vehicles(1).L, vehicles(1).W, vehicles(1).H], ...
'问题 1: 车型 1 单车装箱效果');
saveas(gcf, '问题 1_车型 1_3D.png');
end
% 3D 可视化: 车型 2
if ~isempty(resultsQ1(2).placedItems)
visualize3D(resultsQ1(2).placedItems, ...
[vehicles(2).L, vehicles(2).W, vehicles(2).H], ...
'问题 1: 车型 2 单车装箱效果');
saveas(gcf, '问题 1_车型 2_3D.png');
end
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
% 问题 1(1): 单车型最少车辆
fprintf('\n[Step 3] 求解问题 1(1): 单车型最少车辆...\n');
stepTimer = tic;
resultsQ1_multi1 = solveQ1_multi(goods, vehicles, 1); % 车型 1
resultsQ1_multi2 = solveQ1_multi(goods, vehicles, 2); % 车型 2
% 汇总统计表替代逐车 3D 图
fprintf('\n--- 车型 1 各车装载汇总 ---\n');
fprintf('%-6s %-8s %-14s %-14s\n', '车辆', '件数', '空间利用率', '载重利用率');
for v = 1:resultsQ1_multi1.vehicleCount
m = resultsQ1_multi1.vehicleResults{v}.metrics;
fprintf('第%-4d %-8d %6.2f%% %6.2f%%\n', v, m.numItems, m.spaceUtilization*100,
m.weightUtilization*100);
end
fprintf('\n--- 车型 2 各车装载汇总 ---\n');
fprintf('%-6s %-8s %-14s %-14s\n', '车辆', '件数', '空间利用率', '载重利用率');
for v = 1:resultsQ1_multi2.vehicleCount
m = resultsQ1_multi2.vehicleResults{v}.metrics;
fprintf('第%-4d %-8d %6.2f%% %6.2f%%\n', v, m.numItems, m.spaceUtilization*100,
m.weightUtilization*100);
end
% 利用率对比柱状图
plotMetrics(resultsQ1, resultsQ1_multi1, resultsQ1_multi2);

```

```

saveas(gcf, '问题 1_利用率对比.png');
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
%% ===== 问题 2 =====
fprintf('\n\n%s\n', repmat('=', 1, 50));
fprintf(' 问题 2\n');
fprintf('%s\n', repmat('=', 1, 50));
% 问题 2(1): 总车辆最少
fprintf('\n[Step 4] 求解问题 2(1): 混合车型-总车辆数最少...\n');
stepTimer = tic;
resultsQ2_minVeh = solveQ2_minVehicle(goods, vehicles);
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
% 问题 2(2): 总成本最低
fprintf('\n[Step 5] 求解问题 2(2): 混合车型-总运输成本最低...\n');
stepTimer = tic;
resultsQ2_minCost = solveQ2_minCost(goods, vehicles);
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
% 成本对比图
plotCostCompare(resultsQ2_minVeh, resultsQ2_minCost);
saveas(gcf, '问题 2_成本对比.png');
% 3D 可视化: 问题 2 最优方案的第一辆车
if ~isempty(resultsQ2_minCost.vehicleDetails)
d = resultsQ2_minCost.vehicleDetails{1};
vType = d.vehicleType;
vDim = [vehicles(vType).L, vehicles(vType).W, vehicles(vType).H];
visualize3D(d.placedItems, vDim, ...
sprintf('问题 2: 最低成本方案-第 1 辆车(%s)', d.vehicleName));
saveas(gcf, '问题 2_最优方案_3D.png');
end
%% ===== 问题 3 =====
fprintf('\n\n%s\n', repmat('=', 1, 50));
fprintf(' 问题 3\n');
fprintf('%s\n', repmat('=', 1, 50));
% 参数敏感性分析
fprintf('\n[Step 6] 参数敏感性分析...\n');
stepTimer = tic;

```

```

sensitivityAnalysis(goods, vehicles);
saveas(gcf, '问题 3_敏感性分析.png');
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
%% ===== 附件 2 验证 =====
fprintf('\n\n%s\n', repmat('=', 1, 50));
fprintf(' 附件 2 验证数据集\n');
fprintf('%s\n', repmat('=', 1, 50));
fprintf('\n[Step 7] 附件 2 验证数据集求解...\n');
stepTimer = tic;
[products, vehicleTypes] = loadAttachment2();
resultsAtt2 = solveAttachment2(products, vehicleTypes);
fprintf(' [耗时 %.1f 秒]\n', toc(stepTimer));
%% ===== 结果汇总 =====
fprintf('\n\n%s\n', repmat('=', 1, 50));
fprintf(' 结果汇总\n');
fprintf('%s\n', repmat('=', 1, 50));
fprintf('\n 问题 1(1) 单车装载:\n');
for v = 1:2
    fprintf(' %s: 装载%d 件, 空间利用率%.2f%%, 载重利用率%.2f%%\n', ...
        resultsQ1(v).vehicleName, resultsQ1(v).metrics.numItems, ...
        resultsQ1(v).metrics.spaceUtilization*100, ...
        resultsQ1(v).metrics.weightUtilization*100);
end
fprintf('\n 问题 1(1) 最少车辆:\n');
fprintf(' 车型 1: %d 辆, 成本%d 元\n', resultsQ1_multi1.vehicleCount,
    resultsQ1_multi1.totalCost);
fprintf(' 车型 2: %d 辆, 成本%d 元\n', resultsQ1_multi2.vehicleCount,
    resultsQ1_multi2.totalCost);
fprintf('\n 问题 2(1) 混合车型-最少车辆: %d 辆, 成本%d 元\n', ...
    resultsQ2_minVeh.totalVehicles, resultsQ2_minVeh.totalCost);
fprintf(' 问题 2(2) 混合车型-最低成本: %d 辆, 成本%d 元\n', ...
    resultsQ2_minCost.totalVehicles, resultsQ2_minCost.totalCost);
fprintf('\n===== \n');
fprintf(' 全部求解完成!\n');
fprintf(' 总计耗时: %.1f 秒\n', toc(totalTimer));

```



```

fprintf('=====\\n');
% 保存工作区
save('results.mat');
fprintf('结果已保存到 results.mat\\n');

```

附录 C. 2:

```

function [goods, vehicles] = initData()
% initData - 初始化货物和车型数据
% 输出:
% goods - 货物结构体数组
% vehicles - 车型结构体数组
%% 车型参数
vehicles(1).name = '车型 1(轻型厢式货车)';
vehicles(1).L = 420; % 长 cm
vehicles(1).W = 210; % 宽 cm
vehicles(1).H = 220; % 高 cm
vehicles(1).maxWeight = 6000; % 额定载重 kg
vehicles(1).cost = 450; % 单次运输成本 元/趟

vehicles(2).name = '车型 2(中型厢式货车)';
vehicles(2).L = 680;
vehicles(2).W = 245;
vehicles(2).H = 250;
vehicles(2).maxWeight = 10000;
vehicles(2).cost = 700;
%% 货物参数
% 类型: 1=标准件, 2=易碎件, 3=定向件
goods(1).id = 'G1';
goods(1).type = 1; % 标准件
goods(1).length = 60; % cm
goods(1).width = 40;
goods(1).height = 30;
goods(1).weight = 12; % kg
goods(1).quantity = 800;

```

```
goods(2).id = 'G2';  
goods(2).type = 1; % 标准件  
goods(2).length = 50;  
goods(2).width = 35;  
goods(2).height = 25;  
goods(2).weight = 8;  
goods(2).quantity = 1000;
```

```
goods(3).id = 'G3';  
goods(3).type = 2; % 易碎件  
goods(3).length = 70;  
goods(3).width = 50;  
goods(3).height = 40;  
goods(3).weight = 15;  
goods(3).quantity = 300;
```

```
goods(4).id = 'G4';  
goods(4).type = 3; % 定向件  
goods(4).length = 80;  
goods(4).width = 60;  
goods(4).height = 50;  
goods(4).weight = 25;  
goods(4).quantity = 400;
```

```
goods(5).id = 'G5';  
goods(5).type = 3; % 定向件  
goods(5).length = 40;  
goods(5).width = 40;  
goods(5).height = 60;  
goods(5).weight = 18;  
goods(5).quantity = 500;
```

```
end
```

附录 C. 3:

```

function [placedItems, unplacedItems] = pack3D(itemList, vehicleDim, maxWeight,
sortMode)
% pack3D - 基于极点法的三维装箱核心算法（性能优化版）
% 优化：向量化约束检查、极点数限制、轻量级评分、提前终止
% 输入：
% itemList - 待装货物结构体数组
% vehicleDim - [L, W, H] 车厢内尺寸(cm)
% maxWeight - 车辆额定载重(kg)
% sortMode - 排序模式：'dual'=双目标(密度+体积), 'volume'=纯体积(默认)
% vehicleDim - [L, W, H] 车厢内尺寸(cm)
% maxWeight - 车辆额定载重(kg)
% 输出：
% placedItems - 已放置货物结构体数组(含 pos, dim, type, weight, id)
% unplacedItems - 未能放置的货物结构体数组

safetyGap = 3;
effectiveH = vehicleDim(3) - safetyGap;

if nargin < 4
sortMode = 'volume';
end

%% 初始化
extremePoints = [0, 0, 0];

% 矩阵缓存已放置货物（避免结构体字段访问开销，支持向量化运算）
nItems = length(itemList);

placedItems = struct('id', {}, 'type', {}, 'pos', {}, 'dim', {}, 'weight', {});

% 预分配 unplacedItems 避免逐个增长的  $O(n^2)$  开销
unplacedItems = struct('id', {}, 'type', {}, 'length', {}, 'width', {}, ...
'height', {}, 'weight', {});
unplacedCount = 0;
if nItems > 0

```

```

unplacedItems(nItems) =
struct('id','', 'type',0, 'length',0, 'width',0, 'height',0, 'weight',0);
end

currentWeight = 0;

pPos = zeros(nItems, 3); % 位置 [x, y, z]
pDim = zeros(nItems, 3); % 尺寸 [l, w, h]
pEndC = zeros(nItems, 3); % 终点缓存 = pPos + pDim
pType = zeros(nItems, 1); % 货物类型
pWeight = zeros(nItems, 1); % 重量
nPlaced = 0;

%% 货物排序：标准件/定向件优先，易碎件最后
volumes = zeros(nItems, 1);
itemWeights = zeros(nItems, 1);
priorities = zeros(nItems, 1);
for i = 1:nItems
volumes(i) = itemList(i).length * itemList(i).width * itemList(i).height;
itemWeights(i) = itemList(i).weight;
switch itemList(i).type
case 1, priorities(i) = 2; % 标准件
case 3, priorities(i) = 2; % 定向件
case 2, priorities(i) = 1; % 易碎件最后
end
end
if strcmp(sortMode, 'dual')
% 双目标排序：体积+密度加权，密度高的优先（提高载重利用率）
maxVol = max(volumes);
densities = itemWeights ./ volumes;
maxDensity = max(densities);
compositeScore = 0.4 * (volumes / maxVol) + 0.6 * (densities / maxDensity);
[~, sortIdx] = sortrows([-priorities, -compositeScore]);
else
% 默认：纯体积降序（最大化空间利用率，适用于 FFD 最少车辆）
[~, sortIdx] = sortrows([-priorities, -volumes]);

```

```

end
itemList = itemList(sortIdx);

% 计算平均重量（用于双目标评分中的重量归一化）
weights = zeros(nItems, 1);
for i = 1:nItems
    weights(i) = itemList(i).weight;
end
avgWeight = mean(weights);

MAX_EP = 300; % 极点数上限

% 同规格跳过机制：相同规格物品在相同 nPlaced 下不重复尝试
prevDims = [0 0 0 0];
prevFailed = false;
prevFailNPlaced = -1;
consecutiveFails = 0;

%% 逐个放置货物
for i = 1:nItems
    item = itemList(i);

    % 同规格跳过：若与上次失败物品规格相同且 nPlaced 未变，直接跳过
    curDims = [item.type, item.length, item.width, item.height];
    if isequal(curDims, prevDims) && prevFailed && nPlaced == prevFailNPlaced
        unplacedCount = unplacedCount + 1;
        unplacedItems(unplacedCount) = item;
        continue;
    end
    if ~isequal(curDims, prevDims)
        prevDims = curDims;
        prevFailed = false;
    end

    % 重量预检
    if currentWeight + item.weight > maxWeight

```

```

unplacedCount = unplacedCount + 1;
unplacedItems(unplacedCount) = item;
continue;
end

oris = getOrientations(item.length, item.width, item.height, item.type);
nOri = size(oris, 1);

bestScore = -inf;
bestPos = [];
bestDim = [];
isPlaced = false;

nEP = size(extremePoints, 1);
for ep = 1:nEP
    epPos = extremePoints(ep, :);
    for ori = 1:nOri
        curDim = oris(ori, :);

% 边界快速检查
        if epPos(1) + curDim(1) > vehicleDim(1) + 0.01 || ...
            epPos(2) + curDim(2) > vehicleDim(2) + 0.01 || ...
            epPos(3) + curDim(3) > effectiveH + 0.01
            continue;
        end

% 向量化约束检查
        if nPlaced > 0
            valid = checkFast(epPos, curDim, item.type, item.weight, ...
                pPos(1:nPlaced,:), pDim(1:nPlaced,:), pEndC(1:nPlaced,:), ...
                pType(1:nPlaced), pWeight(1:nPlaced));
        else
            valid = true;
        end

        if valid
            score = scoreFast(epPos, curDim, vehicleDim, effectiveH, ...

```

```

pPos(1:nPlaced,:), pEndC(1:nPlaced,:), nPlaced, ...
item.weight, avgWeight);
if score > bestScore
bestScore = score;
bestPos = epPos;
bestDim = curDim;
isPlaced = true;
end
end
end

% （不使用提前终止，确保最优放置）
end

if isPlaced
newItem.id = item.id;
newItem.type = item.type;
newItem.pos = bestPos;
newItem.dim = bestDim;
newItem.weight = item.weight;
placedItems(end+1) = newItem; %#ok<AGROW>

nPlaced = nPlaced + 1;
pPos(nPlaced, :) = bestPos;
pDim(nPlaced, :) = bestDim;
pEndC(nPlaced, :) = bestPos + bestDim;
pType(nPlaced) = item.type;
pWeight(nPlaced) = item.weight;
currentWeight = currentWeight + item.weight;

prevFailed = false;
prevFailNPlaced = -1;
consecutiveFails = 0;

% 更新极点（含有限投影极点：仅最近 20 个已放置货物）
extremePoints = updateEP(extremePoints, bestPos, bestDim, ...
vehicleDim, effectiveH, pPos(1:nPlaced,:), pEndC(1:nPlaced,:), nPlaced, MAX_EP);

```

```

else
    unplacedCount = unplacedCount + 1;
    unplacedItems(unplacedCount) = item;
    prevFailed = true;
    prevFailNPlaced = nPlaced;
    consecutiveFails = consecutiveFails + 1;
    % 连续失败安全阈值：超过 200 次连续失败则终止
    if nPlaced > 0 && consecutiveFails > 200
        for j = (i+1):nItems
            unplacedCount = unplacedCount + 1;
            unplacedItems(unplacedCount) = itemList(j);
        end
        break;
    end
end

% 裁剪预分配的 unplacedItems
unplacedItems = unplacedItems(1:unplacedCount);

end

%% ===== 内部函数 =====

function valid = checkFast(pos, dim, itemType, itemWeight, pP, pD, pE, pT, pW)
% 向量化约束检查 – 用矩阵运算替代逐个循环
valid = true;
endPos = pos + dim;

% 1. 重叠检查（全向量化 AABB）
sepX = (pos(1) >= pE(:,1) - 0.01) | (endPos(1) <= pP(:,1) + 0.01);
sepY = (pos(2) >= pE(:,2) - 0.01) | (endPos(2) <= pP(:,2) + 0.01);
sepZ = (pos(3) >= pE(:,3) - 0.01) | (endPos(3) <= pP(:,3) + 0.01);
if ~all(sepX | sepY | sepZ)
    valid = false; return;
end

```



```

end

% 2. 支撑面检查（按题目原文分类处理）
if pos(3) > 0.01
topZ = pP(:,3) + pD(:,3);

if itemType == 2 % 易碎件：底面须 100%贴合车厢底面或标准件顶面
% 只能由标准件(type=1)提供支撑
atLevel = abs(topZ - pos(3)) < 0.5 & (pT == 1);
if ~any(atLevel)
valid = false; return;
end
supportArea = 0;
bottomArea = dim(1) * dim(2);
idx = find(atLevel);
for k = 1:length(idx)
j = idx(k);
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
supportArea = supportArea + ox * oy;
end
end
if supportArea / bottomArea < 0.999 % 100%完全贴合
valid = false; return;
end
else % 标准件/定向件：须有正面积接触支撑（物理可行性）
atLevel = abs(topZ - pos(3)) < 0.5;
if ~any(atLevel)
valid = false; return;
end
hasSupport = false;
idx = find(atLevel);
for k = 1:length(idx)
j = idx(k);
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));

```

```

oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
hasSupport = true;
break;
end
end
if ~hasSupport
valid = false; return;
end
end
end

```

% 3. 易碎件约束：不能放在已放置易碎件的正上方

```

fragile = (pT == 2);
if any(fragile)
fIdx = find(fragile);
for k = 1:length(fIdx)
j = fIdx(k);
if pos(3) >= pE(j,3) - 0.5
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
valid = false; return;
end
end
end
end

```

% 4. 定向件重心约束：上层重心须在支撑定向件投影内

```

if pos(3) > 0.01
directed = (pT == 3);
if any(directed)
dIdx = find(directed);
topZ_d = pP(dIdx,3) + pD(dIdx,3);
atLev = abs(topZ_d - pos(3)) < 0.5;
dAtLevel = dIdx(atLev);
cx = pos(1) + dim(1)/2;

```

```

cy = pos(2) + dim(2)/2;
for k = 1:length(dAtLevel)
j = dAtLevel(k);
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
if cx < pP(j,1) || cx > pE(j,1) || cy < pP(j,2) || cy > pE(j,2)
valid = false; return;
end
end
end
end
end

% 5. 承重约束: 下层承重 <= 500 kg/m^2
topZ_all = pP(:,3) + pD(:,3);
supportLev = abs(topZ_all - pos(3)) < 0.5;
sIdx = find(supportLev);
for k = 1:length(sIdx)
j = sIdx(k);
topArea_m2 = pD(j,1) * pD(j,2) / 10000;
if topArea_m2 <= 0, continue; end

topWeight = 0;
aboveZ = pP(:,3) >= pE(j,3) - 0.5;
aIdx = find(aboveZ);
for m = 1:length(aIdx)
ai = aIdx(m);
ox = min(pE(ai,1), pE(j,1)) - max(pP(ai,1), pP(j,1));
oy = min(pE(ai,2), pE(j,2)) - max(pP(ai,2), pP(j,2));
if ox > 0 && oy > 0
topWeight = topWeight + pW(ai);
end
end
topWeight = topWeight + itemWeight;
if topWeight / topArea_m2 > 500
valid = false; return;

```

end

end

end

```
function score = scoreFast(pos, dim, vDim, effH, pP, pE, nP, itemWeight,  
    avgWeight)
```

```
% 评分函数：位置 + 接触面积 + 地面 + 重量贡献（双目标优化）
```

```
% 位置得分：优先底部、靠近原点
```

```
nx = pos(1) / vDim(1);
```

```
ny = pos(2) / vDim(2);
```

```
nz = pos(3) / effH;
```

```
posScore = 1 - (0.3*nx + 0.3*ny + 0.4*nz);
```

```
% 底面贴合得分
```

```
floorScore = double(pos(3) < 0.01);
```

```
% 接触面积得分（向量化计算，与原始算法精度一致）
```

```
contactScore = 0;
```

```
if nP > 0
```

```
    endPos = pos + dim;
```

```
% X 方向接触：左/右面相邻
```

```
touchXL = abs(pos(1) - pE(:,1)) < 0.5;
```

```
touchXR = abs(endPos(1) - pP(:,1)) < 0.5;
```

```
oyS = max(pos(2), pP(:,2));
```

```
oyE = min(endPos(2), pE(:,2));
```

```
ozS = max(pos(3), pP(:,3));
```

```
ozE = min(endPos(3), pE(:,3));
```

```
areaX = max(oyE - oyS, 0) .* max(ozE - ozS, 0) .* (touchXL | touchXR);
```

```
% Y 方向接触：前/后面相邻
```

```
touchYL = abs(pos(2) - pE(:,2)) < 0.5;
```

```
touchYR = abs(endPos(2) - pP(:,2)) < 0.5;
```

```

oxS = max(pos(1), pP(:,1));
oxE = min(endPos(1), pE(:,1));
areaY = max(oxE - oxS, 0) .* max(ozE - ozS, 0) .* (touchYL | touchYR);

% Z 方向接触：上/下面相邻
touchZB = abs(pos(3) - pE(:,3)) < 0.5;
touchZT = abs(endPos(3) - pP(:,3)) < 0.5;
areaZ = max(oxE - oxS, 0) .* max(oyE - oyS, 0) .* (touchZB | touchZT);

totalArea = sum(areaX + areaY + areaZ);
maxArea = 2 * (dim(1)*dim(2) + dim(2)*dim(3) + dim(1)*dim(3));
if maxArea > 0
    contactScore = totalArea / maxArea;
end
end

% 重量贡献得分：鼓励优先放置较重的货物，提高载重利用率
weightScore = itemWeight / avgWeight;
if weightScore > 2, weightScore = 2; end

score = 0.20 * posScore + 0.30 * contactScore + 0.25 * floorScore + 0.25 *
weightScore;

end

function newEPs = updateEP(oldEPs, pos, dim, vDim, effH, pP, pE, nP, maxEP)
% 极点更新：3 个基本极点 + 有限投影极点（最近 20 个已放置货物）

% 生成 3 个新极点
ep1 = [pos(1)+dim(1), pos(2), pos(3)];
ep2 = [pos(1), pos(2)+dim(2), pos(3)];
ep3 = [pos(1), pos(2), pos(3)+dim(3)];

newCandidates = [ep1; ep2; ep3];

```

```

% 投影极点：基于所有已放置货物的不同顶面高度
uniqueZ = unique(pE(1:nP, 3));
for k = 1:length(uniqueZ)
    projEP = [pos(1), pos(2), uniqueZ(k)];
    if projEP(3) > 0 && projEP(3) < effH
        newCandidates = [newCandidates; projEP]; %#ok<AGROW>
    end
end

allEPs = [oldEPs; newCandidates];
nEP = size(allEPs, 1);
n = size(pP, 1);

% 批量有效性检查
validMask = (allEPs(:,1) >= 0) & (allEPs(:,1) < vDim(1)) & ...
    (allEPs(:,2) >= 0) & (allEPs(:,2) < vDim(2)) & ...
    (allEPs(:,3) >= 0) & (allEPs(:,3) < effH);

% 移除在已放置货物内部的极点（逐货物向量化检查）
for j = 1:n
    if ~any(validMask), break; end
    validMask = validMask & ~( ...
        (allEPs(:,1) >= pP(j,1)) & (allEPs(:,1) < pE(j,1)) & ...
        (allEPs(:,2) >= pP(j,2)) & (allEPs(:,2) < pE(j,2)) & ...
        (allEPs(:,3) >= pP(j,3)) & (allEPs(:,3) < pE(j,3)) );
end

newEPs = allEPs(validMask, :);
newEPs = unique(newEPs, 'rows');

% 按 z,y,x 排序（优先低层、靠后、靠右）
if ~isempty(newEPs)
    [~, idx] = sortrows(newEPs, [3, 2, 1]);
    newEPs = newEPs(idx, :);
end

```

```

% 极点数限制
if size(newEPs, 1) > maxEP
newEPs = newEPs(1:maxEP, :);
end

end

function [placedItems, unplacedItems] = pack3D(itemList, vehicleDim, maxWeight,
sortMode)
% pack3D - 基于极点法的三维装箱核心算法（性能优化版）
% 优化：向量化约束检查、极点数限制、轻量级评分、提前终止
% 输入：
% itemList - 待装货物结构体数组
% vehicleDim - [L, W, H] 车厢内尺寸(cm)
% maxWeight - 车辆额定载重(kg)
% sortMode - 排序模式：'dual'=双目标(密度+体积), 'volume'=纯体积(默认)
% vehicleDim - [L, W, H] 车厢内尺寸(cm)
% maxWeight - 车辆额定载重(kg)
% 输出：
% placedItems - 已放置货物结构体数组(含 pos, dim, type, weight, id)
% unplacedItems - 未能放置的货物结构体数组

safetyGap = 3;
effectiveH = vehicleDim(3) - safetyGap;

if nargin < 4
sortMode = 'volume';
end

%% 初始化
extremePoints = [0, 0, 0];

% 矩阵缓存已放置货物（避免结构体字段访问开销，支持向量化运算）
nItems = length(itemList);

```

```

placedItems = struct('id', {}, 'type', {}, 'pos', {}, 'dim', {}, 'weight', {});

% 预分配 unplacedItems 避免逐个增长的 O(n^2)开销
unplacedItems = struct('id', {}, 'type', {}, 'length', {}, 'width', {}, ...
    'height', {}, 'weight', {});
unplacedCount = 0;
if nItems > 0
    unplacedItems(nItems) =
        struct('id', '', 'type', 0, 'length', 0, 'width', 0, 'height', 0, 'weight', 0);
end

currentWeight = 0;

pPos = zeros(nItems, 3); % 位置 [x, y, z]
pDim = zeros(nItems, 3); % 尺寸 [l, w, h]
pEndC = zeros(nItems, 3); % 终点缓存 = pPos + pDim
pType = zeros(nItems, 1); % 货物类型
pWeight = zeros(nItems, 1); % 重量
nPlaced = 0;

%% 货物排序：标准件/定向件优先，易碎件最后
volumes = zeros(nItems, 1);
itemWeights = zeros(nItems, 1);
priorities = zeros(nItems, 1);
for i = 1:nItems
    volumes(i) = itemList(i).length * itemList(i).width * itemList(i).height;
    itemWeights(i) = itemList(i).weight;
    switch itemList(i).type
        case 1, priorities(i) = 2; % 标准件
        case 3, priorities(i) = 2; % 定向件
        case 2, priorities(i) = 1; % 易碎件最后
    end
end
if strcmp(sortMode, 'dual')
    % 双目标排序：体积+密度加权，密度高的优先（提高载重利用率）
    maxVol = max(volumes);

```



```

densities = itemWeights ./ volumes;
maxDensity = max(densities);
compositeScore = 0.4 * (volumes / maxVol) + 0.6 * (densities / maxDensity);
[~, sortIdx] = sortrows([-priorities, -compositeScore]);
else
% 默认：纯体积降序（最大化空间利用率，适用于 FFD 最少车辆）
[~, sortIdx] = sortrows([-priorities, -volumes]);
end
itemList = itemList(sortIdx);

% 计算平均重量（用于双目标评分中的重量归一化）
weights = zeros(nItems, 1);
for i = 1:nItems
weights(i) = itemList(i).weight;
end
avgWeight = mean(weights);

MAX_EP = 300; % 极点数上限

% 同规格跳过机制：相同规格物品在相同 nPlaced 下不重复尝试
prevDims = [0 0 0 0];
prevFailed = false;
prevFailNPlaced = -1;
consecutiveFails = 0;

%% 逐个放置货物
for i = 1:nItems
item = itemList(i);

% 同规格跳过：若与上次失败物品规格相同且 nPlaced 未变，直接跳过
curDims = [item.type, item.length, item.width, item.height];
if isequal(curDims, prevDims) && prevFailed && nPlaced == prevFailNPlaced
unplacedCount = unplacedCount + 1;
unplacedItems(unplacedCount) = item;
continue;
end

```

```

if ~isequal(curDims, prevDims)
prevDims = curDims;
prevFailed = false;
end

% 重量预检
if currentWeight + item.weight > maxWeight
unplacedCount = unplacedCount + 1;
unplacedItems(unplacedCount) = item;
continue;
end

oris = getOrientations(item.length, item.width, item.height, item.type);
nOri = size(oris, 1);

bestScore = -inf;
bestPos = [];
bestDim = [];
isPlaced = false;

nEP = size(extremePoints, 1);
for ep = 1:nEP
epPos = extremePoints(ep, :);
for ori = 1:nOri
curDim = oris(ori, :);

% 边界快速检查
if epPos(1) + curDim(1) > vehicleDim(1) + 0.01 || ...
epPos(2) + curDim(2) > vehicleDim(2) + 0.01 || ...
epPos(3) + curDim(3) > effectiveH + 0.01
continue;
end

% 向量化约束检查
if nPlaced > 0
valid = checkFast(epPos, curDim, item.type, item.weight, ...
pPos(1:nPlaced,:), pDim(1:nPlaced,:), pEndC(1:nPlaced,:), ...

```

```

pType(1:nPlaced), pWeight(1:nPlaced));
else
valid = true;
end

if valid
score = scoreFast(epPos, curDim, vehicleDim, effectiveH, ...
pPos(1:nPlaced,:), pEndC(1:nPlaced,:), nPlaced, ...
item.weight, avgWeight);
if score > bestScore
bestScore = score;
bestPos = epPos;
bestDim = curDim;
isPlaced = true;
end
end
end

% （不使用提前终止，确保最优放置）
end

if isPlaced
newItem.id = item.id;
newItem.type = item.type;
newItem.pos = bestPos;
newItem.dim = bestDim;
newItem.weight = item.weight;
placedItems(end+1) = newItem; %#ok<AGROW>

nPlaced = nPlaced + 1;
pPos(nPlaced, :) = bestPos;
pDim(nPlaced, :) = bestDim;
pEndC(nPlaced, :) = bestPos + bestDim;
pType(nPlaced) = item.type;
pWeight(nPlaced) = item.weight;
currentWeight = currentWeight + item.weight;

```

```

prevFailed = false;
prevFailNPlaced = -1;
consecutiveFails = 0;

% 更新极点（含有限投影极点：仅最近 20 个已放置货物）
extremePoints = updateEP(extremePoints, bestPos, bestDim, ...
vehicleDim, effectiveH, pPos(1:nPlaced,:), pEndC(1:nPlaced,:), nPlaced, MAX_EP);
else
unplacedCount = unplacedCount + 1;
unplacedItems(unplacedCount) = item;
prevFailed = true;
prevFailNPlaced = nPlaced;
consecutiveFails = consecutiveFails + 1;
% 连续失败安全阈值：超过 200 次连续失败则终止
if nPlaced > 0 && consecutiveFails > 200
for j = (i+1):nItems
unplacedCount = unplacedCount + 1;
unplacedItems(unplacedCount) = itemList(j);
end
break;
end
end
end

% 裁剪预分配的 unplacedItems
unplacedItems = unplacedItems(1:unplacedCount);

end

%% ===== 内部函数 =====

function valid = checkFast(pos, dim, itemType, itemWeight, pP, pD, pE, pT, pW)
% 向量化约束检查 – 用矩阵运算替代逐个循环
valid = true;
endPos = pos + dim;

```

% 1. 重叠检查（全向量化 AABB）

```
sepX = (pos(1) >= pE(:,1) - 0.01) | (endPos(1) <= pP(:,1) + 0.01);  
sepY = (pos(2) >= pE(:,2) - 0.01) | (endPos(2) <= pP(:,2) + 0.01);  
sepZ = (pos(3) >= pE(:,3) - 0.01) | (endPos(3) <= pP(:,3) + 0.01);  
if ~all(sepX | sepY | sepZ)  
    valid = false; return;  
end
```

% 2. 支撑面检查（按题目原文分类处理）

```
if pos(3) > 0.01  
    topZ = pP(:,3) + pD(:,3);  
  
    if itemType == 2 % 易碎件：底面须 100%贴合车厢底面或标准件顶面  
        % 只能由标准件(type=1)提供支撑  
        atLevel = abs(topZ - pos(3)) < 0.5 & (pT == 1);  
        if ~any(atLevel)  
            valid = false; return;  
        end  
        supportArea = 0;  
        bottomArea = dim(1) * dim(2);  
        idx = find(atLevel);  
        for k = 1:length(idx)  
            j = idx(k);  
            ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));  
            oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));  
            if ox > 0 && oy > 0  
                supportArea = supportArea + ox * oy;  
            end  
        end  
        if supportArea / bottomArea < 0.999 % 100%完全贴合  
            valid = false; return;  
        end  
    else % 标准件/定向件：须有正面积接触支撑（物理可行性）  
        atLevel = abs(topZ - pos(3)) < 0.5;  
        if ~any(atLevel)
```

```

valid = false; return;
end
hasSupport = false;
idx = find(atLevel);
for k = 1:length(idx)
j = idx(k);
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
hasSupport = true;
break;
end
end
if ~hasSupport
valid = false; return;
end
end
end

```

% 3. 易碎件约束：不能放在已放置易碎件的正上方

```

fragile = (pT == 2);
if any(fragile)
fIdx = find(fragile);
for k = 1:length(fIdx)
j = fIdx(k);
if pos(3) >= pE(j,3) - 0.5
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
valid = false; return;
end
end
end
end

```

% 4. 定向件重心约束：上层重心须在支撑定向件投影内

```

if pos(3) > 0.01

```

```

directed = (pT == 3);
if any(directed)
dIdx = find(directed);
topZ_d = pP(dIdx,3) + pD(dIdx,3);
atLev = abs(topZ_d - pos(3)) < 0.5;
dAtLevel = dIdx(atLev);
cx = pos(1) + dim(1)/2;
cy = pos(2) + dim(2)/2;
for k = 1:length(dAtLevel)
j = dAtLevel(k);
ox = min(endPos(1), pE(j,1)) - max(pos(1), pP(j,1));
oy = min(endPos(2), pE(j,2)) - max(pos(2), pP(j,2));
if ox > 0 && oy > 0
if cx < pP(j,1) || cx > pE(j,1) || cy < pP(j,2) || cy > pE(j,2)
valid = false; return;
end
end
end
end
end

```

% 5. 承重约束: 下层承重 <= 500 kg/m²

```

topZ_all = pP(:,3) + pD(:,3);
supportLev = abs(topZ_all - pos(3)) < 0.5;
sIdx = find(supportLev);
for k = 1:length(sIdx)
j = sIdx(k);
topArea_m2 = pD(j,1) * pD(j,2) / 10000;
if topArea_m2 <= 0, continue; end

topWeight = 0;
aboveZ = pP(:,3) >= pE(j,3) - 0.5;
aIdx = find(aboveZ);
for m = 1:length(aIdx)
ai = aIdx(m);
ox = min(pE(ai,1), pE(j,1)) - max(pP(ai,1), pP(j,1));
oy = min(pE(ai,2), pE(j,2)) - max(pP(ai,2), pP(j,2));

```

```

if ox > 0 && oy > 0
topWeight = topWeight + pw(ai);
end
end
topWeight = topWeight + itemWeight;
if topWeight / topArea_m2 > 500
valid = false; return;
end
end

end

function score = scoreFast(pos, dim, vDim, effH, pP, pE, nP, itemWeight,
avgWeight)
% 评分函数：位置 + 接触面积 + 地面 + 重量贡献（双目标优化）

% 位置得分：优先底部、靠近原点
nx = pos(1) / vDim(1);
ny = pos(2) / vDim(2);
nz = pos(3) / effH;
posScore = 1 - (0.3*nx + 0.3*ny + 0.4*nz);

% 底面贴合得分
floorScore = double(pos(3) < 0.01);

% 接触面积得分（向量化计算，与原始算法精度一致）
contactScore = 0;
if nP > 0
endPos = pos + dim;

% X 方向接触：左/右面相邻
touchXL = abs(pos(1) - pE(:,1)) < 0.5;
touchXR = abs(endPos(1) - pP(:,1)) < 0.5;
oyS = max(pos(2), pP(:,2));
oyE = min(endPos(2), pE(:,2));

```



```

ozS = max(pos(3), pP(:,3));
ozE = min(endPos(3), pE(:,3));
areaX = max(oyE - oyS, 0) .* max(ozE - ozS, 0) .* (touchXL | touchXR);

% Y 方向接触：前/后面相邻
touchYL = abs(pos(2) - pE(:,2)) < 0.5;
touchYR = abs(endPos(2) - pP(:,2)) < 0.5;
oxS = max(pos(1), pP(:,1));
oxE = min(endPos(1), pE(:,1));
areaY = max(oxE - oxS, 0) .* max(oyE - oyS, 0) .* (touchYL | touchYR);

% Z 方向接触：上/下面相邻
touchZB = abs(pos(3) - pE(:,3)) < 0.5;
touchZT = abs(endPos(3) - pP(:,3)) < 0.5;
areaZ = max(oxE - oxS, 0) .* max(oyE - oyS, 0) .* (touchZB | touchZT);

totalArea = sum(areaX + areaY + areaZ);
maxArea = 2 * (dim(1)*dim(2) + dim(2)*dim(3) + dim(1)*dim(3));
if maxArea > 0
    contactScore = totalArea / maxArea;
end
end

% 重量贡献得分：鼓励优先放置较重的货物，提高载重利用率
weightScore = itemWeight / avgWeight;
if weightScore > 2, weightScore = 2; end

score = 0.20 * posScore + 0.30 * contactScore + 0.25 * floorScore + 0.25 *
weightScore;

end

function newEPs = updateEP(oldEPs, pos, dim, vDim, effH, pP, pE, nP, maxEP)
% 极点更新：3 个基本极点 + 有限投影极点（最近 20 个已放置货物）

```

```

% 生成 3 个新极点
ep1 = [pos(1)+dim(1), pos(2), pos(3)];
ep2 = [pos(1), pos(2)+dim(2), pos(3)];
ep3 = [pos(1), pos(2), pos(3)+dim(3)];

newCandidates = [ep1; ep2; ep3];

% 投影极点：基于所有已放置货物的不同顶面高度
uniqueZ = unique(pE(1:nP, 3));
for k = 1:length(uniqueZ)
    projEP = [pos(1), pos(2), uniqueZ(k)];
    if projEP(3) > 0 && projEP(3) < effH
        newCandidates = [newCandidates; projEP]; %#ok<AGROW>
    end
end

allEPs = [oldEPs; newCandidates];
nEP = size(allEPs, 1);
n = size(pP, 1);

% 批量有效性检查
validMask = (allEPs(:,1) >= 0) & (allEPs(:,1) < vDim(1)) & ...
    (allEPs(:,2) >= 0) & (allEPs(:,2) < vDim(2)) & ...
    (allEPs(:,3) >= 0) & (allEPs(:,3) < effH);

% 移除在已放置货物内部的极点（逐货物向量化检查）
for j = 1:n
    if ~any(validMask), break; end
    validMask = validMask & ~( ...
        (allEPs(:,1) >= pP(j,1)) & (allEPs(:,1) < pE(j,1)) & ...
        (allEPs(:,2) >= pP(j,2)) & (allEPs(:,2) < pE(j,2)) & ...
        (allEPs(:,3) >= pP(j,3)) & (allEPs(:,3) < pE(j,3)) );
end

newEPs = allEPs(validMask, :);
newEPs = unique(newEPs, 'rows');

```

```

% 按 z,y,x 排序（优先低层、靠后、靠右）
if ~isempty(newEPs)
[~, idx] = sortrows(newEPs, [3, 2, 1]);
newEPs = newEPs(idx, :);
end

% 极点数限制
if size(newEPs, 1) > maxEP
newEPs = newEPs(1:maxEP, :);
end

end

```

附录 C.4:

```

function itemList = expandGoods(goods)
% expandGoods - 将货物类型列表展开为单件列表（预分配优化）

totalItems = sum([goods.quantity]);
itemList(totalItems) = struct('id','', 'type',0, 'length',0, 'width',0, ...
'height',0, 'weight',0);
idx = 0;
for i = 1:length(goods)
for j = 1:goods(i).quantity
idx = idx + 1;
itemList(idx).id = sprintf('%s_%d', goods(i).id, j);
itemList(idx).type = goods(i).type;
itemList(idx).length = goods(i).length;
itemList(idx).width = goods(i).width;
itemList(idx).height = goods(i).height;
itemList(idx).weight = goods(i).weight;
end
end

end

```

附录C.5:

```
function oris = getOrientations(l, w, h, itemType)
% getOrientations - 根据货物类型返回所有合法的摆放姿态
% 输入:
% l, w, h - 原始长宽高
% itemType - 1=标准件, 2=易碎件, 3=定向件
% 输出:
% oris - N×3 矩阵, 每行为一种姿态的[长, 宽, 高]

switch itemType
case 1 % 标准件: 6 种摆放姿态
    allOris = [l w h; l h w; w l h; w h l; h l w; h w l];
    % 去重
    oris = unique(allOris, 'rows');

case 2 % 易碎件: 6 种姿态 (但不可堆叠约束在放置时处理)
    allOris = [l w h; l h w; w l h; w h l; h l w; h w l];
    oris = unique(allOris, 'rows');

case 3 % 定向件: 仅一种固定姿态
    oris = [l w h];

otherwise
    oris = [l w h];
end

end
```

附录C.6:

```
function [valid, msg] = checkConstraints(pos, dim, itemType, itemWeight, ...
placedItems, vehicleDim, safetyGap)
% checkConstraints - 综合约束检查
% 输入:
% pos - [x, y, z] 待放置货物的位置
% dim - [l, w, h] 待放置货物的尺寸(当前姿态)
```

```

% itemType - 1=标准件, 2=易碎件, 3=定向件
% itemWeight - 货物重量(kg)
% placedItems- 已放置货物结构体数组(含 pos, dim, type, weight 字段)
% vehicleDim - [L, W, H] 车厢内尺寸
% safetyGap - 顶部安全间隙(cm), 默认 3
% 输出:
% valid - 逻辑值
% msg - 约束违反说明(若 valid=false)

if nargin < 7
    safetyGap = 3;
end

valid = true;
msg = '';

%% 1. 边界检查
if pos(1) < 0 || pos(2) < 0 || pos(3) < 0
    valid = false; msg = '坐标为负'; return;
end
if pos(1) + dim(1) > vehicleDim(1) + 0.01
    valid = false; msg = '超出车厢长度'; return;
end
if pos(2) + dim(2) > vehicleDim(2) + 0.01
    valid = false; msg = '超出车厢宽度'; return;
end
if pos(3) + dim(3) > vehicleDim(3) - safetyGap + 0.01
    valid = false; msg = '超出车厢高度(含安全间隙)'; return;
end

%% 2. 重叠检查
nPlaced = length(placedItems);
for i = 1:nPlaced
    if checkOverlap(pos, dim, placedItems(i).pos, placedItems(i).dim)
        valid = false; msg = sprintf('与第%d 件货物重叠', i); return;
    end
end

```

```

end
end

%% 3. 支撑面检查（按题目原文分类处理）
if pos(3) > 0.01 % 不在地面上
if itemType == 2 % 易碎件：底面须 100%贴合车厢底面或标准件顶面
supportArea = 0;
itemBottomArea = dim(1) * dim(2);

for i = 1:nPlaced
pi = placedItems(i);
if pi.type ~= 1, continue; end % 仅标准件可支撑易碎件
if abs(pi.pos(3) + pi.dim(3) - pos(3)) < 0.5
ox1 = max(pos(1), pi.pos(1));
ox2 = min(pos(1)+dim(1), pi.pos(1)+pi.dim(1));
oy1 = max(pos(2), pi.pos(2));
oy2 = min(pos(2)+dim(2), pi.pos(2)+pi.dim(2));
if ox2 > ox1 && oy2 > oy1
supportArea = supportArea + (ox2 - ox1) * (oy2 - oy1);
end
end
end

supportRatio = supportArea / itemBottomArea;
if supportRatio < 0.999 % 100%完全贴合
valid = false; msg = sprintf('易碎件底面支撑不足(%.1f%%)', supportRatio*100);
return;
end
else % 标准件/定向件：须有正面积接触支撑
hasSupport = false;
for i = 1:nPlaced
pi = placedItems(i);
if abs(pi.pos(3) + pi.dim(3) - pos(3)) < 0.5
ox1 = max(pos(1), pi.pos(1));
ox2 = min(pos(1)+dim(1), pi.pos(1)+pi.dim(1));
oy1 = max(pos(2), pi.pos(2));

```

```

oy2 = min(pos(2)+dim(2), pi.pos(2)+pi.dim(2));
if ox2 > ox1 && oy2 > oy1
hasSupport = true;
break;
end
end
end
if ~hasSupport
valid = false; msg = '非地面货物无支撑面'; return;
end
end
end

%% 4. 易碎件约束：已放置的易碎件上方不可再放货物
for i = 1:nPlaced
if placedItems(i).type == 2 % 已放置的是易碎件
pi = placedItems(i);
% 检查当前货物是否在该易碎件的正上方
if pos(3) >= pi.pos(3) + pi.dim(3) - 0.5 % 在其上方
% 检查 x-y 投影是否有重叠
ox1 = max(pos(1), pi.pos(1));
ox2 = min(pos(1)+dim(1), pi.pos(1)+pi.dim(1));
oy1 = max(pos(2), pi.pos(2));
oy2 = min(pos(2)+dim(2), pi.pos(2)+pi.dim(2));
if ox2 > ox1 && oy2 > oy1
valid = false; msg = '在易碎件上方放置货物'; return;
end
end
end
end

%% 5. 定向件重心约束：上层货物重心不得超出其实际支撑的定向件投影
if pos(3) > 0.01 % 不在地面
% 只检查实际支撑当前货物的定向件(即有 x-y 投影重叠的)
for i = 1:nPlaced
if placedItems(i).type == 3 % 下方的定向件

```

```

pi = placedItems(i);
if abs(pi.pos(3) + pi.dim(3) - pos(3)) < 0.5
% 先检查是否有 x-y 投影重叠(即该定向件是否实际支撑当前货物)
ox1 = max(pos(1), pi.pos(1));
ox2 = min(pos(1)+dim(1), pi.pos(1)+pi.dim(1));
oy1 = max(pos(2), pi.pos(2));
oy2 = min(pos(2)+dim(2), pi.pos(2)+pi.dim(2));
if ox2 > ox1 && oy2 > oy1 % 有重叠, 即该定向件支撑当前货物
% 检查当前货物重心是否在该定向件投影范围内
cx = pos(1) + dim(1)/2;
cy = pos(2) + dim(2)/2;
if cx < pi.pos(1) || cx > pi.pos(1) + pi.dim(1) || ...
cy < pi.pos(2) || cy > pi.pos(2) + pi.dim(2)
valid = false; msg = '上层货物重心超出定向件投影'; return;
end
end
end
end
end
end

%% 6. 承重约束: 下层承重 <= 500 kg/m^2
% 检查当前货物下方的已放置货物的承重是否超标
for i = 1:nPlaced
pi = placedItems(i);
if abs(pi.pos(3) + pi.dim(3) - pos(3)) < 0.5 % 当前货物直接放在它上面
% 计算该已放置货物顶面的总承重
topWeight = 0;
topArea = pi.dim(1) * pi.dim(2) / 10000; % 转换为 m^2

% 统计已放在该货物上方的所有货物重量
for j = 1:nPlaced
pj = placedItems(j);
if pj.pos(3) >= pi.pos(3) + pi.dim(3) - 0.5
ox1 = max(pj.pos(1), pi.pos(1));
ox2 = min(pj.pos(1)+pj.dim(1), pi.pos(1)+pi.dim(1));

```



```

oy1 = max(pj.pos(2), pi.pos(2));
oy2 = min(pj.pos(2)+pj.dim(2), pi.pos(2)+pi.dim(2));
if ox2 > ox1 && oy2 > oy1
topWeight = topWeight + pj.weight;
end
end
end
% 加上当前要放置的货物重量
topWeight = topWeight + itemWeight;

if topArea > 0 && topWeight / topArea > 500
valid = false; msg = sprintf('承重超标(%.1f kg/m^2)', topWeight/topArea); return;
End
cend

end

```

附录 C.7:

```

function overlap = checkOverlap(pos1, dim1, pos2, dim2)
% checkOverlap - AABB 碰撞检测，检查两个长方体是否重叠
% 输入:
% pos1 - [x1, y1, z1] 第一个长方体的左后下角坐标
% dim1 - [l1, w1, h1] 第一个长方体的尺寸
% pos2 - [x2, y2, z2] 第二个长方体的左后下角坐标
% dim2 - [l2, w2, h2] 第二个长方体的尺寸
% 输出:
% overlap - 逻辑值, true 表示重叠

% 在三个轴向上分别检查是否有间隔
% 若任一轴向上没有重叠, 则两箱体不重叠
overlapX = (pos1(1) < pos2(1) + dim2(1)) && (pos2(1) < pos1(1) + dim1(1));
overlapY = (pos1(2) < pos2(2) + dim2(2)) && (pos2(2) < pos1(2) + dim1(2));
overlapZ = (pos1(3) < pos2(3) + dim2(3)) && (pos2(3) < pos1(3) + dim1(3));

overlap = overlapX && overlapY && overlapZ;

```

end

附录 C. 8:

```
function metrics = calcMetrics(placedItems, vehicleDim, maxWeight)
% calcMetrics - 计算装箱利用率指标
% 输入:
% placedItems - 已放置货物结构体数组
% vehicleDim - [L, W, H] 车厢内尺寸
% maxWeight - 车辆额定载重
% 输出:
% metrics - 结构体, 含空间利用率/载重利用率等

vehicleVolume = vehicleDim(1) * vehicleDim(2) * vehicleDim(3);

totalItemVolume = 0;
totalItemWeight = 0;
nItems = length(placedItems);

for i = 1:nItems
totalItemVolume = totalItemVolume + ...
placedItems(i).dim(1) * placedItems(i).dim(2) * placedItems(i).dim(3);
totalItemWeight = totalItemWeight + placedItems(i).weight;
end

metrics.numItems = nItems;
metrics.totalVolume = totalItemVolume;
metrics.vehicleVolume = vehicleVolume;
metrics.spaceUtilization = totalItemVolume / vehicleVolume; % 空间利用率
metrics.totalWeight = totalItemWeight;
metrics.maxWeight = maxWeight;
metrics.weightUtilization = totalItemWeight / maxWeight; % 载重利用率

end
```

附录 C. 9:

```

function results = solveQ1_single(goods, vehicles)
% solveQ1_single - 问题 1(1): 分别以车型 1、车型 2 装载, 单车满载率最大化
% 输入:
% goods - 货物结构体数组
% vehicles - 车型结构体数组
% 输出:
% results - 各车型的装箱结果

fprintf('\n===== \n');
fprintf(' 问题 1(1): 单车满载率最大化 \n');
fprintf('===== \n');

results = struct();

for v = 1:2
    veh = vehicles(v);
    vehicleDim = [veh.L, veh.W, veh.H];

    fprintf('\n--- %s --- \n', veh.name);
    fprintf('车厢尺寸: %d x %d x %d cm \n', veh.L, veh.W, veh.H);
    fprintf('额定载重: %d kg \n', veh.maxWeight);

    % 展开所有货物为单件列表
    itemList = expandGoods(goods);

    % 调用 3D 装箱算法 (双目标模式: 同时优化满容率和满载率)
    [placedItems, unplacedItems] = pack3D(itemList, vehicleDim, veh.maxWeight,
    'dual');

    % 计算利用率指标
    metrics = calcMetrics(placedItems, vehicleDim, veh.maxWeight);

    % 输出结果
    fprintf('放置货物数: %d / %d \n', metrics.numItems, length(itemList));
    fprintf('空间利用率: %.2f%% \n', metrics.spaceUtilization * 100);

```

```

fprintf('载重利用率: %.2f%%\n', metrics.weightUtilization * 100);
fprintf('货物总体积: %.0f cm^3\n', metrics.totalVolume);
fprintf('车厢总体积: %.0f cm^3\n', metrics.vehicleVolume);
fprintf('货物总重量: %.1f kg\n', metrics.totalWeight);

% 输出放置方案表（限制显示前 20 件）
fprintf('\n 放置方案:\n');
fprintf('%-8s %-8s %-20s %-20s\n', '货物编号', '类型', '放置坐标(x,y,z)', '尺寸
(l,w,h)');
fprintf('%s\n', repmat('-', 1, 60));
maxPrint = 20;
for i = 1:min(maxPrint, length(placedItems))
pi = placedItems(i);
typeStr = getTypeStr(pi.type);
fprintf('%-8s %-8s (%6.1f,%6.1f,%6.1f) (%3.0f,%3.0f,%3.0f)\n', ...
pi.id, typeStr, pi.pos(1), pi.pos(2), pi.pos(3), ...
pi.dim(1), pi.dim(2), pi.dim(3));
end
if length(placedItems) > maxPrint
fprintf(' ... 省略 %d 件 ...\n', length(placedItems) - maxPrint);
end

results(v).vehicleName = veh.name;
results(v).placedItems = placedItems;
results(v).unplacedItems = unplacedItems;
results(v).metrics = metrics;
end

end

```

附录 C.10:

```

function results = solveQ1_multi(goods, vehicles, vehicleTypeIdx)
% solveQ1_multi - 问题 1(1): 单一车型装完所有货物, 车辆数最少(FFD)
% 输入:
% goods - 货物结构体数组

```

```

% vehicles - 车型结构体数组
% vehicleTypeIdx - 使用的车型索引(1 或 2)
% 输出:
% results - 多车装载结果

veh = vehicles(vehicleTypeIdx);
vehicleDim = [veh.L, veh.W, veh.H];

fprintf('\n=====');
fprintf(' 问题 1(1): 单车型最少车辆 - %s\n', veh.name);
fprintf('=====');

% 展开所有货物为单件列表
itemList = expandGoods(goods);
totalItems = length(itemList);

% 按体积降序排序
volumes = zeros(totalItems, 1);
priorities = zeros(totalItems, 1);
for i = 1:totalItems
    volumes(i) = itemList(i).length * itemList(i).width * itemList(i).height;
    switch itemList(i).type
        case 2, priorities(i) = 3;
        case 3, priorities(i) = 2;
        case 1, priorities(i) = 1;
    end
end
[~, sortIdx] = sortrows([-priorities, -volumes]);
remainingItems = itemList(sortIdx);

% FFD: 逐车装载
vehicleResults = {};
vehicleCount = 0;

while ~isempty(remainingItems)
    vehicleCount = vehicleCount + 1;

```

```

fprintf('\n--- 第%d 辆车 ---\n', vehicleCount);

[placedItems, unplacedItems] = pack3D(remainingItems, vehicleDim, veh.maxWeight);

metrics = calcMetrics(placedItems, vehicleDim, veh.maxWeight);
fprintf('装载: %d 件, 空间利用率: %.2f%%, 载重利用率: %.2f%%\n', ...
metrics.numItems, metrics.spaceUtilization*100, metrics.weightUtilization*100);

vehicleResults{vehicleCount}.placedItems = placedItems; %#ok<AGROW>
vehicleResults{vehicleCount}.metrics = metrics; %#ok<AGROW>

remainingItems = unplacedItems;

% 安全检查: 如果没有放置任何货物, 防止死循环
if isempty(placedItems)
fprintf('警告: 有%d 件货物无法装入任何车辆!\n', length(remainingItems));
break;
end
end

fprintf('\n=== 总计 ===\n');
fprintf('使用%s 共 %d 辆\n', veh.name, vehicleCount);
fprintf('总成本: %d 元\n', vehicleCount * veh.cost);

% 输出前 3 辆车的详细放置方案 (每辆最多 20 件)
maxShowVehicles = 3;
maxPrint = 20;
for v = 1:min(maxShowVehicles, vehicleCount)
fprintf('\n--- 第%d 辆车放置方案 ---\n', v);
fprintf('%-8s %-8s %-20s %-20s\n', '货物编号', '类型', '放置坐标(x,y,z)', '尺寸
(1,w,h)');
fprintf('%s\n', repmat('-', 1, 60));
pi_arr = vehicleResults{v}.placedItems;
for i = 1:min(maxPrint, length(pi_arr))
pi = pi_arr(i);
typeStr = getTypeStr(pi.type);

```

```

fprintf('%-8s %-8s (%6.1f,%6.1f,%6.1f) (%3.0f,%3.0f,%3.0f)\n', ...
pi.id, typeStr, pi.pos(1), pi.pos(2), pi.pos(3), ...
pi.dim(1), pi.dim(2), pi.dim(3));
end
if length(pi_arr) > maxPrint
fprintf(' ... 省略 %d 件 ...\n', length(pi_arr) - maxPrint);
end
end
if vehicleCount > maxShowVehicles
fprintf('\n(仅显示前%d 辆车方案, 共%d 辆)\n', maxShowVehicles, vehicleCount);
end

results.vehicleName = veh.name;
results.vehicleCount = vehicleCount;
results.vehicleResults = vehicleResults;
results.totalCost = vehicleCount * veh.cost;

end

```

附录 C.11:

```

function results = solveQ2_minVehicle(goods, vehicles)
% solveQ2_minVehicle - 问题 2(1): 混合车型, 总运输车辆最少
% 枚举多种组合策略, 选择总车辆数最少的方案

fprintf('\n=====');
fprintf(' 问题 2(1): 混合车型-总车辆数最少\n');
fprintf('=====');

itemList = expandGoods(goods);

strategies = {'全用车型 1', '全用车型 2', '先大后小', '先小后大'};
bestVehicles = inf;
bestResult = [];

for s = 1:4
remaining = itemList;

```

```

vDetails = {};
t1Count = 0;
t2Count = 0;
cost = 0;

switch s
case 1 % 全用车型 1
veh = vehicles(1);
vDim = [veh.L, veh.W, veh.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim, veh.maxWeight);
if isempty(placed), break; end
t1Count = t1Count + 1;
m = calcMetrics(placed, vDim, veh.maxWeight);
d.vehicleType = 1; d.vehicleName = veh.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh.cost;
remaining = unplaced;
end

case 2 % 全用车型 2
veh = vehicles(2);
vDim = [veh.L, veh.W, veh.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim, veh.maxWeight);
if isempty(placed), break; end
t2Count = t2Count + 1;
m = calcMetrics(placed, vDim, veh.maxWeight);
d.vehicleType = 2; d.vehicleName = veh.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh.cost;
remaining = unplaced;
end

case 3 % 先大后小（大车装到装不下再切小车）

```



```

veh2 = vehicles(2);
vDim2 = [veh2.L, veh2.W, veh2.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim2, veh2.maxWeight);
    if isempty(placed), break; end
    t2Count = t2Count + 1;
    m = calcMetrics(placed, vDim2, veh2.maxWeight);
    d.vehicleType = 2; d.vehicleName = veh2.name;
    d.placedItems = placed; d.metrics = m;
    vDetails{end+1} = d; %#ok<AGROW>
    cost = cost + veh2.cost;
    remaining = unplaced;
end
veh1 = vehicles(1);
vDim1 = [veh1.L, veh1.W, veh1.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim1, veh1.maxWeight);
    if isempty(placed), break; end
    t1Count = t1Count + 1;
    m = calcMetrics(placed, vDim1, veh1.maxWeight);
    d.vehicleType = 1; d.vehicleName = veh1.name;
    d.placedItems = placed; d.metrics = m;
    vDetails{end+1} = d; %#ok<AGROW>
    cost = cost + veh1.cost;
    remaining = unplaced;
end

case 4 % 先小后大
veh1 = vehicles(1);
vDim1 = [veh1.L, veh1.W, veh1.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim1, veh1.maxWeight);
    if isempty(placed), break; end
    t1Count = t1Count + 1;
    m = calcMetrics(placed, vDim1, veh1.maxWeight);
    d.vehicleType = 1; d.vehicleName = veh1.name;
    d.placedItems = placed; d.metrics = m;

```

```

vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh1.cost;
remaining = unplaced;
end
veh2 = vehicles(2);
vDim2 = [veh2.L, veh2.W, veh2.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim2, veh2.maxWeight);
if isempty(placed), break; end
t2Count = t2Count + 1;
m = calcMetrics(placed, vDim2, veh2.maxWeight);
d.vehicleType = 2; d.vehicleName = veh2.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh2.cost;
remaining = unplaced;
end
end

totalVeh = t1Count + t2Count;
fprintf('策略%d(%s): 车型 1=%d, 车型 2=%d, 总车辆=%d, 成本=%d 元\n', ...
s, strategies{s}, t1Count, t2Count, totalVeh, cost);

% 选择总车辆数最少的; 车辆数相同时选成本更低的
if isempty(remaining) && (totalVeh < bestVehicles || ...
(totalVeh == bestVehicles && cost < bestResult.totalCost))
bestVehicles = totalVeh;
bestResult.vehicleDetails = vDetails;
bestResult.totalVehicles = totalVeh;
bestResult.totalCost = cost;
bestResult.type1Count = t1Count;
bestResult.type2Count = t2Count;
bestResult.strategy = strategies{s};
end
end

results = bestResult;

```

```

fprintf('\n=== 最少车辆方案结果 ===\n');
fprintf('最优策略: %s\n', results.strategy);
fprintf('车型 1 数量: %d\n', results.type1Count);
fprintf('车型 2 数量: %d\n', results.type2Count);
fprintf('总车辆数: %d\n', results.totalVehicles);
fprintf('总成本: %d 元\n', results.totalCost);

```

end

附录 C.12:

```

function results = solveQ2_minCost(goods, vehicles)
% solveQ2_minCost - 问题 2(2): 混合车型, 总运输成本最低
% 比较不同车型组合策略, 选择总成本最低的方案

fprintf('\n===== \n');
fprintf(' 问题 2(2): 混合车型-总运输成本最低\n');
fprintf('===== \n');

itemList = expandGoods(goods);

% 策略: 尝试多种组合, 选择成本最低的
% 方案 1: 全用车型 1
% 方案 2: 全用车型 2
% 方案 3: 先车型 2 后车型 1 (和 minVehicle 类似)
% 方案 4: 先车型 1 后车型 2
% 方案 5: 交替使用

strategies = {'全用车型 1', '全用车型 2', '先大后小', '先小后大'};
bestCost = inf;
bestResult = [];

for s = 1:4
    remaining = itemList;
    vDetails = {};

```

```

t1Count = 0;
t2Count = 0;
cost = 0;

switch s
case 1 % 全用车型 1
veh = vehicles(1);
vDim = [veh.L, veh.W, veh.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim, veh.maxWeight);
if isempty(placed), break; end
t1Count = t1Count + 1;
m = calcMetrics(placed, vDim, veh.maxWeight);
d.vehicleType = 1; d.vehicleName = veh.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh.cost;
remaining = unplaced;
end

case 2 % 全用车型 2
veh = vehicles(2);
vDim = [veh.L, veh.W, veh.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim, veh.maxWeight);
if isempty(placed), break; end
t2Count = t2Count + 1;
m = calcMetrics(placed, vDim, veh.maxWeight);
d.vehicleType = 2; d.vehicleName = veh.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh.cost;
remaining = unplaced;
end

case 3 % 先大后小（大车装到装不下再切小车）
veh2 = vehicles(2);

```

```

vDim2 = [veh2.L, veh2.W, veh2.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim2, veh2.maxWeight);
    if isempty(placed), break; end
    t2Count = t2Count + 1;
    m = calcMetrics(placed, vDim2, veh2.maxWeight);
    d.vehicleType = 2; d.vehicleName = veh2.name;
    d.placedItems = placed; d.metrics = m;
    vDetails{end+1} = d; %#ok<AGROW>
    cost = cost + veh2.cost;
    remaining = unplaced;
end
veh1 = vehicles(1);
vDim1 = [veh1.L, veh1.W, veh1.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim1, veh1.maxWeight);
    if isempty(placed), break; end
    t1Count = t1Count + 1;
    m = calcMetrics(placed, vDim1, veh1.maxWeight);
    d.vehicleType = 1; d.vehicleName = veh1.name;
    d.placedItems = placed; d.metrics = m;
    vDetails{end+1} = d; %#ok<AGROW>
    cost = cost + veh1.cost;
    remaining = unplaced;
end

case 4 % 先小后大
veh1 = vehicles(1);
vDim1 = [veh1.L, veh1.W, veh1.H];
while ~isempty(remaining)
    [placed, unplaced] = pack3D(remaining, vDim1, veh1.maxWeight);
    if isempty(placed), break; end
    t1Count = t1Count + 1;
    m = calcMetrics(placed, vDim1, veh1.maxWeight);
    d.vehicleType = 1; d.vehicleName = veh1.name;
    d.placedItems = placed; d.metrics = m;
    vDetails{end+1} = d; %#ok<AGROW>

```

```

cost = cost + veh1.cost;
remaining = unplaced;
end
veh2 = vehicles(2);
vDim2 = [veh2.L, veh2.W, veh2.H];
while ~isempty(remaining)
[placed, unplaced] = pack3D(remaining, vDim2, veh2.maxWeight);
if isempty(placed), break; end
t2Count = t2Count + 1;
m = calcMetrics(placed, vDim2, veh2.maxWeight);
d.vehicleType = 2; d.vehicleName = veh2.name;
d.placedItems = placed; d.metrics = m;
vDetails{end+1} = d; %#ok<AGROW>
cost = cost + veh2.cost;
remaining = unplaced;
end
end

fprintf('策略%d(%s): 车型 1=%d, 车型 2=%d, 总车辆=%d, 成本=%d 元\n', ...
s, strategies{s}, t1Count, t2Count, t1Count+t2Count, cost);

if cost < bestCost && isempty(remaining)
bestCost = cost;
bestResult.vehicleDetails = vDetails;
bestResult.totalVehicles = t1Count + t2Count;
bestResult.totalCost = cost;
bestResult.type1Count = t1Count;
bestResult.type2Count = t2Count;
bestResult.strategy = strategies{s};
end
end

results = bestResult;

fprintf('\n=== 最低成本方案结果 ===\n');
fprintf('最优策略: %s\n', results.strategy);

```

```

fprintf('车型 1 数量: %d\n', results.type1Count);
fprintf('车型 2 数量: %d\n', results.type2Count);
fprintf('总车辆数: %d\n', results.totalVehicles);
fprintf('总成本: %d 元\n', results.totalCost);

```

end

附录 C.13:

```

function sensitivityAnalysis(goods, vehicles)
% sensitivityAnalysis - 参数敏感性分析入口（优化版：减少数据点）

fprintf('\n===== \n');
fprintf(' 问题 3: 参数敏感性分析\n');
fprintf('===== \n');

% 调用可视化敏感性分析
plotSensitivity(goods, vehicles);

%% 数值分析：输出详细敏感性数据
fprintf('\n--- 货物尺寸变化影响分析 --- \n');

sizeScales = [0.8, 1.0, 1.2]; % 3 个数据点
fprintf('尺寸比例\t空间利用率(车型 1)\t空间利用率(车型 2)\n');

for s = 1:length(sizeScales)
    scaledGoods = goods;
    for g = 1:length(scaledGoods)
        scaledGoods(g).length = round(goods(g).length * sizeScales(s));
        scaledGoods(g).width = round(goods(g).width * sizeScales(s));
        scaledGoods(g).height = round(goods(g).height * sizeScales(s));
    end

    itemList = expandGoods(scaledGoods);

% 车型 1

```

```

[placed1, ~] = pack3D(itemList, [vehicles(1).L, vehicles(1).W, vehicles(1).H],
vehicles(1).maxWeight);
m1 = calcMetrics(placed1, [vehicles(1).L, vehicles(1).W, vehicles(1).H],
vehicles(1).maxWeight);

% 车型 2
[placed2, ~] = pack3D(itemList, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
vehicles(2).maxWeight);
m2 = calcMetrics(placed2, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
vehicles(2).maxWeight);

fprintf('%0.1f\t\t%0.2f%\t\t%0.2f%\n', sizeScales(s), m1.spaceUtilization*100,
m2.spaceUtilization*100);
end

fprintf('\n--- 承重限制变化影响分析 ---\n');
fprintf('(承重上限影响主要体现在堆叠层数限制上)\n');

fprintf('\n 敏感性分析完成。 \n');

end

```

附录 C. 14:

```

function plotSensitivity(goods, vehicles)
% plotSensitivity - 参数敏感性分析曲线图（优化版：减少数据点，合并循环）

fprintf('\n===== \n');
fprintf(' 参数敏感性分析 \n');
fprintf('===== \n');

figure('Name', '参数敏感性分析', 'NumberTitle', 'off', 'Position', [100 100 1200
800]);

%% 合并分析：货物数量变化对空间利用率、车辆数、成本的影响
scales = [0.5, 1.0, 1.5]; % 3 个数据点
nScales = length(scales);

```



```

spaceUtil1 = zeros(nScales, 1);
spaceUtil2 = zeros(nScales, 1);
vehicleCounts1 = zeros(nScales, 1);
vehicleCounts2 = zeros(nScales, 1);

for s = 1:nScales
    fprintf(' [敏感性分析] 数量倍数 %.1f 进行中...\n', scales(s));
    scaledGoods = goods;
    for g = 1:length(scaledGoods)
        scaledGoods(g).quantity = round(goods(g).quantity * scales(s));
    end

    itemList = expandGoods(scaledGoods);

    % 车型 1 单车
    [placed, ~] = pack3D(itemList, [vehicles(1).L, vehicles(1).W, vehicles(1).H],
        vehicles(1).maxWeight);
    m = calcMetrics(placed, [vehicles(1).L, vehicles(1).W, vehicles(1).H],
        vehicles(1).maxWeight);
    spaceUtil1(s) = m.spaceUtilization * 100;

    % 车型 2 单车
    [placed, ~] = pack3D(itemList, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
        vehicles(2).maxWeight);
    m = calcMetrics(placed, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
        vehicles(2).maxWeight);
    spaceUtil2(s) = m.spaceUtilization * 100;

    fprintf(' 单车: 车型 1=%.1f%%, 车型 2=%.1f%%\n', spaceUtil1(s), spaceUtil2(s));

    % 车型 1 FFD
    remaining = itemList;
    count = 0;
    while ~isempty(remaining)
        count = count + 1;
    end
end

```

```

[placed, remaining] = pack3D(remaining, ...
[vehicles(1).L, vehicles(1).W, vehicles(1).H], vehicles(1).maxWeight);
if isempty(placed), break; end
end
vehicleCounts1(s) = count;

% 车型 2 FFD
remaining = itemList;
count = 0;
while ~isempty(remaining)
count = count + 1;
[placed, remaining] = pack3D(remaining, ...
[vehicles(2).L, vehicles(2).W, vehicles(2).H], vehicles(2).maxWeight);
if isempty(placed), break; end
end
vehicleCounts2(s) = count;

fprintf(' FFD: 车型 1=%d 辆, 车型 2=%d 辆\n', vehicleCounts1(s), vehicleCounts2(s));
end

%% 1. 货物数量 vs 空间利用率
subplot(2, 2, 1);
plot(scales, spaceUtil1, 'b-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot(scales, spaceUtil2, 'r-s', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
hold off;
xlabel('货物数量倍数');
ylabel('空间利用率 (%)');
title('货物数量 vs 空间利用率');
legend('车型 1', '车型 2', 'Location', 'best');
grid on;

%% 2. 货物数量 vs 所需车辆数
subplot(2, 2, 2);
plot(scales, vehicleCounts1, 'b-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;

```

```

plot(scales, vehicleCounts2, 'r-s', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
hold off;
xlabel('货物数量倍数');
ylabel('所需车辆数');
title('货物数量 vs 所需车辆数');
legend('车型 1', '车型 2', 'Location', 'best');
grid on;

```

%% 3. 货物数量 vs 运输成本

```

subplot(2, 2, 3);
costs1 = vehicleCounts1 * vehicles(1).cost;
costs2 = vehicleCounts2 * vehicles(2).cost;
plot(scales, costs1, 'b-o', 'LineWidth', 1.5, 'MarkerFaceColor', 'b');
hold on;
plot(scales, costs2, 'r-s', 'LineWidth', 1.5, 'MarkerFaceColor', 'r');
hold off;
xlabel('货物数量倍数');
ylabel('总运输成本 (元)');
title('货物数量 vs 运输成本');
legend('车型 1', '车型 2', 'Location', 'best');
grid on;

```

%% 4. 易碎件比例 vs 空间利用率

```

subplot(2, 2, 4);

fragileRatios = [0, 0.2, 0.5]; % 3 个数据点
spaceUtils = zeros(length(fragileRatios), 1);

totalQty = sum([goods.quantity]);
for fr = 1:length(fragileRatios)
    fprintf(' [敏感性分析] 易碎件占比 %.0f%% 进行中...\n', fragileRatios(fr)*100);
    modGoods = goods;
    fragileQty = round(totalQty * fragileRatios(fr));
    standardQty = totalQty - fragileQty;

    modGoods(1).quantity = round(standardQty * 0.45);

```

```

modGoods(2).quantity = round(standardQty * 0.55);
modGoods(3).quantity = fragileQty;
modGoods(4).quantity = round(totalQty * 0.13);
modGoods(5).quantity = round(totalQty * 0.17);

itemList = expandGoods(modGoods);
[placed, ~] = pack3D(itemList, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
vehicles(2).maxWeight);
m = calcMetrics(placed, [vehicles(2).L, vehicles(2).W, vehicles(2).H],
vehicles(2).maxWeight);
spaceUtils(fr) = m.spaceUtilization * 100;
end

plot(fragileRatios*100, spaceUtils, 'g-d', 'LineWidth', 1.5, 'MarkerFaceColor',
'g', 'MarkerSize', 8);
xlabel('易碎件占比 (%)');
ylabel('空间利用率 (%)');
title('易碎件比例 vs 空间利用率(车型 2)');
grid on;

sgtitle('参数敏感性分析', 'FontSize', 15, 'FontWeight', 'bold');

fprintf('敏感性分析图表完成\n');

end

```

附录 C. 15:

```

function visualize3D(placedItems, vehicleDim, titleStr)
% visualize3D - 3D 装箱效果可视化
% 输入:
% placedItems - 已放置货物结构体数组
% vehicleDim - [L, W, H] 车厢尺寸
% titleStr - 图标题

if nargin < 3
titleStr = '3D 装箱效果图';

```

```

end

figure('Name', titleStr, 'NumberTitle', 'off', 'Position', [100 100 900 700]);
hold on;

% 绘制车厢轮廓(半透明)
drawBox([0 0 0], vehicleDim, [0.8 0.8 0.8], 0.1, '--');

% 货物颜色方案
colors = struct();
colors.standard = [0.2 0.6 1.0]; % 蓝色 - 标准件
colors.fragile = [1.0 0.3 0.3]; % 红色 - 易碎件
colors.directed = [0.3 0.8 0.3]; % 绿色 - 定向件

nItems = length(placedItems);
for i = 1:nItems
    item = placedItems(i);
    switch item.type
    case 1, c = colors.standard;
    case 2, c = colors.fragile;
    case 3, c = colors.directed;
    otherwise, c = [0.5 0.5 0.5];
    end

    % 添加轻微随机色彩变化以区分同类货物
    c = c + 0.1 * (rand(1,3) - 0.5);
    c = max(0, min(1, c));

    drawBox(item.pos, item.dim, c, 0.7, '-');
end

% 设置坐标轴
xlabel('X - 长度 (cm)', 'FontSize', 12);
ylabel('Y - 宽度 (cm)', 'FontSize', 12);
zlabel('Z - 高度 (cm)', 'FontSize', 12);
title(titleStr, 'FontSize', 14, 'FontWeight', 'bold');

```

```

axis equal;
xlim([-10 vehicleDim(1)+10]);
ylim([-10 vehicleDim(2)+10]);
zlim([-10 vehicleDim(3)+10]);

view(3);
grid on;
light('Position', [1 1 1]);
lighting gouraud;

% 图例
h1 = patch(NaN, NaN, NaN, colors.standard, 'FaceAlpha', 0.7);
h2 = patch(NaN, NaN, NaN, colors.fragile, 'FaceAlpha', 0.7);
h3 = patch(NaN, NaN, NaN, colors.directed, 'FaceAlpha', 0.7);
legend([h1 h2 h3], {'标准件', '易碎件', '定向件'}, ...
'Location', 'northeast', 'FontSize', 11);

hold off;

end

function drawBox(pos, dim, color, alpha, lineStyle)
% 绘制一个长方体
% pos = [x, y, z], dim = [l, w, h]

x = pos(1); y = pos(2); z = pos(3);
l = dim(1); w = dim(2); h = dim(3);

% 8 个顶点
vertices = [
x y z;
x+l y z;
x+l y+w z;
x y+w z;

```

```

x y z+h;
x+1 y z+h;
x+1 y+w z+h;
x y+w z+h;
];

% 6 个面
faces = [
1 2 6 5; % 前面
2 3 7 6; % 右面
3 4 8 7; % 后面
4 1 5 8; % 左面
1 2 3 4; % 底面
5 6 7 8; % 顶面
];

patch('Vertices', vertices, 'Faces', faces, ...
'FaceColor', color, 'FaceAlpha', alpha, ...
'EdgeColor', 'k', 'LineStyle', lineStyle, 'LineWidth', 0.5);

end

```

附录 C.16:

```

function plotMetrics(resultsQ1, resultsQ1_multi1, resultsQ1_multi2)
% plotMetrics - 利用率对比柱状图
% 输入:
% resultsQ1 - 问题 1 单车结果
% resultsQ1_multi1 - 问题 1 多车结果(车型 1)
% resultsQ1_multi2 - 问题 1 多车结果(车型 2)

figure('Name', '利用率对比', 'NumberTitle', 'off', 'Position', [100 100 1000
500]);

%% 子图 1: 单车空间利用率和载重利用率
subplot(1, 2, 1);

```

```

categories = {'车型 1-空间', '车型 1-载重', '车型 2-空间', '车型 2-载重'};
values = [resultsQ1(1).metrics.spaceUtilization * 100, ...
resultsQ1(1).metrics.weightUtilization * 100, ...
resultsQ1(2).metrics.spaceUtilization * 100, ...
resultsQ1(2).metrics.weightUtilization * 100];

barColors = [0.2 0.6 1.0; 0.2 0.6 1.0; 0.3 0.8 0.3; 0.3 0.8 0.3];
b = bar(values);
b.FaceColor = 'flat';
b.CData = barColors;
set(gca, 'XTickLabel', categories, 'FontSize', 10);
ylabel('利用率 (%)', 'FontSize', 12);
title('单车装载利用率对比', 'FontSize', 13, 'FontWeight', 'bold');
ylim([0 100]);
grid on;

% 在柱状图上方显示数值
for i = 1:length(values)
text(i, values(i)+2, sprintf('%.1f%', values(i)), ...
'HorizontalAlignment', 'center', 'FontSize', 10, 'FontWeight', 'bold');
end

%% 子图 2: 多车方案车辆数和成本
subplot(1, 2, 2);
categories2 = {'车型 1', '车型 2'};
vehicleCounts = [resultsQ1_multi1.vehicleCount, resultsQ1_multi2.vehicleCount];
costs = [resultsQ1_multi1.totalCost, resultsQ1_multi2.totalCost];

yyaxis left;
b1 = bar(1:2, vehicleCounts, 0.4, 'FaceColor', [0.2 0.6 1.0]);
ylabel('车辆数', 'FontSize', 12);

yyaxis right;
hold on;
plot(1:2, costs, 'ro-', 'LineWidth', 2, 'MarkerSize', 8, 'MarkerFaceColor', 'r');
ylabel('总成本 (元)', 'FontSize', 12);

```



```

hold off;

set(gca, 'XTick', 1:2, 'XTickLabel', categories2, 'FontSize', 10);
title('多车方案: 车辆数与成本', 'FontSize', 13, 'FontWeight', 'bold');
legend({'车辆数', '总成本'}, 'Location', 'northwest');
grid on;

sgtitle('问题 1: 装载利用率与车辆配置对比', 'FontSize', 15, 'FontWeight', 'bold');

end

```

附录 C.17:

```

function plotCostCompare(resultsMinVeh, resultsMinCost)
% plotCostCompare - 问题 2 两种目标下的成本对比图
% 输入:
% resultsMinVeh - 最少车辆方案结果
% resultsMinCost - 最低成本方案结果

figure('Name', '问题 2 结果对比', 'NumberTitle', 'off', 'Position', [100 100 1000
600]);

%% 子图 1: 车辆数和车型配对比
subplot(1, 2, 1);

data = [resultsMinVeh.type1Count, resultsMinVeh.type2Count;
resultsMinCost.type1Count, resultsMinCost.type2Count];

b = bar(data, 'stacked');
b(1).FaceColor = [0.2 0.6 1.0];
b(2).FaceColor = [0.3 0.8 0.3];

set(gca, 'XTickLabel', {'最少车辆', '最低成本'}, 'FontSize', 11);
ylabel('车辆数量', 'FontSize', 12);
title('车辆配置对比', 'FontSize', 13, 'FontWeight', 'bold');
legend('车型 1(轻型)', '车型 2(中型)', 'Location', 'northwest');

```

```

grid on;

% 在柱状图上显示总数
for i = 1:2
total = data(i,1) + data(i,2);
text(i, total + 0.3, sprintf('共%d 辆', total), ...
'HorizontalAlignment', 'center', 'FontSize', 11, 'FontWeight', 'bold');
end

%% 子图 2: 成本对比
subplot(1, 2, 2);

costs = [resultsMinVeh.totalCost; resultsMinCost.totalCost];
costBreakdown = [resultsMinVeh.type1Count * 450, resultsMinVeh.type2Count * 700;
resultsMinCost.type1Count * 450, resultsMinCost.type2Count * 700];

b2 = bar(costBreakdown, 'stacked');
b2(1).FaceColor = [0.2 0.6 1.0];
b2(2).FaceColor = [0.3 0.8 0.3];

set(gca, 'XTickLabel', {'最少车辆', '最低成本'}, 'FontSize', 11);
ylabel('运输成本 (元)', 'FontSize', 12);
title('运输成本分解对比', 'FontSize', 13, 'FontWeight', 'bold');
legend('车型 1 成本', '车型 2 成本', 'Location', 'northwest');
grid on;

% 显示总成本
for i = 1:2
text(i, costs(i) + 50, sprintf('%d 元', costs(i)), ...
'HorizontalAlignment', 'center', 'FontSize', 11, 'FontWeight', 'bold', 'Color',
'r');
end

% 计算成本节省
if costs(2) < costs(1)
saving = costs(1) - costs(2);

```

```

savingPct = saving / costs(1) * 100;
fprintf('\n 成本优化方案相比车辆最少方案节省: %d 元 (%.1f%%)\n', saving, savingPct);
end

sgtitle('问题 2: 最少车辆 vs 最低成本 对比分析', 'FontSize', 15, 'FontWeight',
'bold');

end

```

附录 C.18:

```

function results = solveAttachment2(products, vehicleTypes)
% solveAttachment2 - 用附件 2 数据验证模型通用性
% 输入:
% products - 产品结构体数组
% vehicleTypes - 车型结构体数组
% 输出:
% results - 验证结果

fprintf('\n=====');
fprintf(' 附件 2: 验证数据集求解\n');
fprintf('=====');

% 将产品转为标准 goods 格式
goods = struct();
for i = 1:length(products)
goods(i).id = sprintf('P%d', i);
goods(i).type = products(i).type;
goods(i).length = products(i).length;
goods(i).width = products(i).width;
goods(i).height = products(i).height;
goods(i).weight = products(i).weight;
goods(i).quantity = products(i).quantity;
end

% 选择几种典型车型进行测试

```

```

testVehicleIndices = [6, 8]; % 6.8 箱货, 17 箱货 (代表性车型)
results = {};

for vi = 1:length(testVehicleIndices)
    vIdx = testVehicleIndices(vi);
    veh = vehicleTypes(vIdx);
    vehicleDim = [veh.L, veh.W, veh.H];

    fprintf('\n--- 测试车型: %s ---\n', veh.name);
    fprintf('尺寸: %.0f x %.0f x %.0f cm\n', veh.L, veh.W, veh.H);

    % 展开货物
    itemList = expandGoods(goods);

    % 使用 FFD 装载
    remainingItems = itemList;
    vehicleResults = {};
    vehicleCount = 0;

    while ~isempty(remainingItems)
        vehicleCount = vehicleCount + 1;
        [placed, unplaced] = pack3D(remainingItems, vehicleDim, veh.maxWeight);

        if isempty(placed)
            fprintf('警告: 有%d 件货物无法装入!\n', length(remainingItems));
            break;
        end

        metrics = calcMetrics(placed, vehicleDim, veh.maxWeight);
        vehicleResults{vehicleCount}.placed = placed;
        vehicleResults{vehicleCount}.metrics = metrics;
        remainingItems = unplaced;
    end

    fprintf('使用 %d 辆车\n', vehicleCount);
    totalCost = vehicleCount * veh.cost;

```

```

fprintf('总运输费用: %.0f 元/1000km\n', totalCost);

% 平均利用率
avgSpace = 0;
avgWeight = 0;
for vc = 1:vehicleCount
    avgSpace = avgSpace + vehicleResults{vc}.metrics.spaceUtilization;
    avgWeight = avgWeight + vehicleResults{vc}.metrics.weightUtilization;
end
avgSpace = avgSpace / vehicleCount * 100;
avgWeight = avgWeight / vehicleCount * 100;
fprintf('平均空间利用率: %.2f%%\n', avgSpace);
fprintf('平均载重利用率: %.2f%%\n', avgWeight);

results{vi}.vehicleName = veh.name;
results{vi}.vehicleCount = vehicleCount;
results{vi}.totalCost = totalCost;
results{vi}.avgSpaceUtil = avgSpace;
results{vi}.avgWeightUtil = avgWeight;
results{vi}.vehicleResults = vehicleResults;
end

end

```

附录 C.19:

```

function [products, vehicleTypes] = loadAttachment2(xlsxPath)
% loadAttachment2 - 读取附件 2 验证数据集
% 输入:
%.xlsxPath - 附件 2.xlsx 文件路径
% 输出:
% products - 箱装产品尺寸结构体数组
% vehicleTypes - 车型尺寸结构体数组

if nargin < 1
   .xlsxPath = 'C:\Users\lx\Desktop\附件 2: 验证数据集.xlsx';
end

```

```

%% 读取箱装产品尺寸
fprintf('读取附件 2 数据...\n');

[numData1, txtData1, rawData1] = xlsread(xlsxPath, '箱装产品尺寸');
[numData2, txtData2, rawData2] = xlsread(xlsxPath, '车型尺寸');

%% 解析产品数据
% 产品名称在第一列，从第 3 行开始(跳过标题)
products = struct();
productNames = {'3.68L*4', '1.8L*6', '4L*4', '5L*4(异形瓶)', '5L*4(常规品)', ...
'8L*2', '10L*2', '20L(软)'};

% 使用仓库测量值(更接近实际)
% 列顺序：名称，化验长，仓库长，化验宽，仓库宽，化验高，仓库高，备注
productData = [
301, 301, 347; % 3.68L*4 (仓库测量值, mm)
346, 237, 314; % 1.8L*6 (取中值)
317, 265, 375; % 4L*4
345, 345, 395; % 5L*4(异形瓶)
322, 322, 378; % 5L*4(常规品)
434, 225, 365; % 8L*2
437, 225, 429; % 10L*2
306, 177, 493; % 20L(软)
];

for i = 1:length(productNames)
products(i).name = productNames{i};
products(i).length = productData(i, 1) / 10; % mm -> cm
products(i).width = productData(i, 2) / 10;
products(i).height = productData(i, 3) / 10;
products(i).type = 1; % 默认标准件
products(i).weight = 10; % 默认重量(kg)，实际应根据产品调整
products(i).quantity = 50; % 默认数量
end

```

```

%% 解析车型数据
vehicleNames = {'单节柜','集装箱','超高柜','4.2 米箱货','6.2 米箱货',...
'6.8 米箱货','9.6 米箱货','17 米箱货',...
'4.2 低栏板','4.2 高栏','6 米 8 高栏','6 米 8 低栏板',...
'9 米 6 高栏','9 米 6 低栏板','13 米高栏','13 米高低板'};

% [长 cm, 宽 cm, 高 cm, 费用元/1000km]
vehicleData = [
585, 235, 239, 3625; % 单节柜
1200, 235, 239, 2000; % 集装箱
1200, 235, 269, 3900; % 超高柜
405, 185, 183, 2800; % 4.2 米箱货(取中值)
595, 195, 190, 4500; % 6.2 米箱货
670, 240, 220, 5000; % 6.8 米箱货
945, 240, 220, 6500; % 9.6 米箱货
1685, 240, 220, 11000; % 17 米箱货
410, 210, 50, 3500; % 4.2 低栏板
410, 210, 200, 4000; % 4.2 高栏
670, 240, 200, 5000; % 6 米 8 高栏
670, 240, 50, 4500; % 6 米 8 低栏板
945, 240, 200, 7000; % 9 米 6 高栏
945, 240, 50, 5000; % 9 米 6 低栏板
1296, 240, 200, 9000; % 13 米高栏
1296, 240, 200, 8800; % 13 米高低板
];

for i = 1:length(vehicleNames)
vehicleTypes(i).name = vehicleNames{i};
vehicleTypes(i).L = vehicleData(i, 1);
vehicleTypes(i).W = vehicleData(i, 2);
vehicleTypes(i).H = vehicleData(i, 3);
vehicleTypes(i).cost = vehicleData(i, 4);
vehicleTypes(i).maxWeight = 10000; % 默认载重

```

```
end
```

```
fprintf('读取完成: %d 种产品, %d 种车型\n', length(products), length(vehicleTypes));
```

```
end
```

附录 C. 20:

```
function str = getTypeStr(typeNum)
```

```
% getTypeStr - 类型编号转中文名称
```

```
switch typeNum
```

```
case 1, str = '标准件';
```

```
case 2, str = '易碎件';
```

```
case 3, str = '定向件';
```

```
otherwise, str = '未知';
```

```
end
```

```
end
```

附录 C. 21:

```
function [bestSolution, bestFitness, fitnessHistory] = GA_optimize(goods,
```

```
vehicles, objectiveType)
```

```
% GA_optimize - 遗传算法优化多车型货物分配
```

```
% 输入:
```

```
% goods - 货物结构体数组
```

```
% vehicles - 车型结构体数组
```

```
% objectiveType - 1=最少车辆, 2=最低成本
```

```
% 输出:
```

```
% bestSolution - 最优分配方案
```

```
% bestFitness - 最优适应度值
```

```
% fitnessHistory - 每代最优适应度
```

```
%% GA 参数
```

```
popSize = 50; % 种群大小
```

```
maxGen = 200; % 最大迭代次数
```



```

crossRate = 0.8; % 交叉率
mutRate = 0.1; % 变异率
eliteRate = 0.1; % 精英保留率

% 展开货物为单件列表
itemList = expandGoods(goods);
nItems = length(itemList);
nVehicleTypes = length(vehicles);

% 估算所需最大车辆数
totalVolume = 0;
for i = 1:nItems
    totalVolume = totalVolume + itemList(i).length * itemList(i).width *
    itemList(i).height;
end
minVehVol = min([vehicles.L] .* [vehicles.W] .* [vehicles.H]);
maxVehicles = ceil(totalVolume / minVehVol * 1.5);
maxVehicles = max(maxVehicles, 5);

%% 初始化种群
% 染色体: nItems 长度的向量, 每个基因值 = 车辆编号(1~maxVehicles)
% 另一个向量: 车辆类型(1 或 2)
population = cell(popSize, 1);
for p = 1:popSize
    chr.assignment = randi(maxVehicles, nItems, 1);
    chr.vehicleTypes = randi(nVehicleTypes, maxVehicles, 1);
    population{p} = chr;
end

%% 设置收敛曲线图
figure('Name', 'GA 收敛曲线', 'NumberTitle', 'off');
hLine = animatedline('Color', 'b', 'LineWidth', 1.5);
xlabel('迭代次数');
if objectiveType == 1
    ylabel('车辆数');

```

```

title('遗传算法收敛曲线 - 最少车辆目标');
else
ylabel('总成本(元)');
title('遗传算法收敛曲线 - 最低成本目标');
end
grid on;

fitnessHistory = zeros(maxGen, 1);

%% 迭代优化
for gen = 1:maxGen
% 计算适应度
fitness = zeros(popSize, 1);
for p = 1:popSize
fitness(p) = evaluateFitness(population{p}, itemList, vehicles, objectiveType);
end

% 记录最优
[bestFit, bestIdx] = min(fitness);
fitnessHistory(gen) = bestFit;

% 更新收敛曲线
addpoints(hLine, gen, bestFit);
drawnow limitrate;

if mod(gen, 20) == 0
fprintf('第%d 代: 最优适应度 = %.2f\n', gen, bestFit);
end

% 精英保留
nElite = max(1, round(popSize * eliteRate));
[~, sortedIdx] = sort(fitness);
newPop = cell(popSize, 1);
for i = 1:nElite
newPop{i} = population{sortedIdx(i)};
end

```

```

% 选择+交叉+变异
for i = (nElite+1):popSize
% 锦标赛选择
p1 = tournamentSelect(population, fitness);
p2 = tournamentSelect(population, fitness);

% 交叉
if rand < crossRate
child = crossover(p1, p2, nItems, maxVehicles);
else
child = p1;
end

% 变异
if rand < mutRate
child = mutate(child, nItems, maxVehicles, nVehicleTypes);
end

newPop{i} = child;
end

population = newPop;
end

% 最终评估
fitness = zeros(popSize, 1);
for p = 1:popSize
fitness(p) = evaluateFitness(population{p}, itemList, vehicles, objectiveType);
end
[bestFitness, bestIdx] = min(fitness);
bestSolution = population{bestIdx};

fprintf('\nGA 优化完成: 最优适应度 = %.2f\n', bestFitness);

end

```

```

function cost = evaluateFitness(chromosome, itemList, vehicles, objectiveType)
% 评估染色体的适应度（越小越好）

nItems = length(itemList);
assignment = chromosome.assignment;
vehicleTypes = chromosome.vehicleTypes;

% 统计每辆车分配的货物
usedVehicles = unique(assignment);
totalCost = 0;
totalVehicles = 0;
penalty = 0;

for vi = 1:length(usedVehicles)
vIdx = usedVehicles(vi);
vType = vehicleTypes(min(vIdx, length(vehicleTypes)));
veh = vehicles(vType);
vehicleDim = [veh.L, veh.W, veh.H];

% 获取分配到该车的货物
itemIndices = find(assignment == vIdx);
if isempty(itemIndices)
continue;
end

subItems = itemList(itemIndices);

% 尝试装箱
[placed, unplaced] = pack3D(subItems, vehicleDim, veh.maxWeight);

if ~isempty(unplaced)
% 惩罚：未能装入的货物
penalty = penalty + length(unplaced) * 10000;
end

```

```

totalVehicles = totalVehicles + 1;
totalCost = totalCost + veh.cost;
end

```

```

if objectiveType == 1
cost = totalVehicles + penalty;
else
cost = totalCost + penalty;
end

end

```

```

function selected = tournamentSelect(population, fitness)
% 锦标赛选择 (k=3)
k = 3;
n = length(population);
candidates = randi(n, k, 1);
[~, best] = min(fitness(candidates));
selected = population{candidates(best)};
end

```

```

function child = crossover(p1, p2, nItems, maxVehicles)
% 均匀交叉
child.assignment = p1.assignment;
child.vehicleTypes = p1.vehicleTypes;
mask = rand(nItems, 1) > 0.5;
child.assignment(mask) = p2.assignment(mask);
maskV = rand(maxVehicles, 1) > 0.5;
child.vehicleTypes(maskV) = p2.vehicleTypes(maskV);
end

```

```

function chr = mutate(chr, nItems, maxVehicles, nVehicleTypes)
% 变异：随机改变几个货物的分配

```

```
nMut = max(1, round(nItems * 0.05));  
mutIdx = randi(nItems, nMut, 1);  
chr.assignment(mutIdx) = randi(maxVehicles, nMut, 1);  
  
% 车型变异  
if rand < 0.3  
    vMut = randi(maxVehicles);  
    chr.vehicleTypes(vMut) = randi(nVehicleTypes);  
end  
end
```

